CS254 – Network Security

Crypto intro

Jan 27, 2025

Logistics

- This Wed: proposal presentation
- Attack & tool presentation next two Weds (Feb 7 and Feb 14)
- In-class quiz about crypto (next Thursday or afterwards)

Today's Class

Essential Cryptography

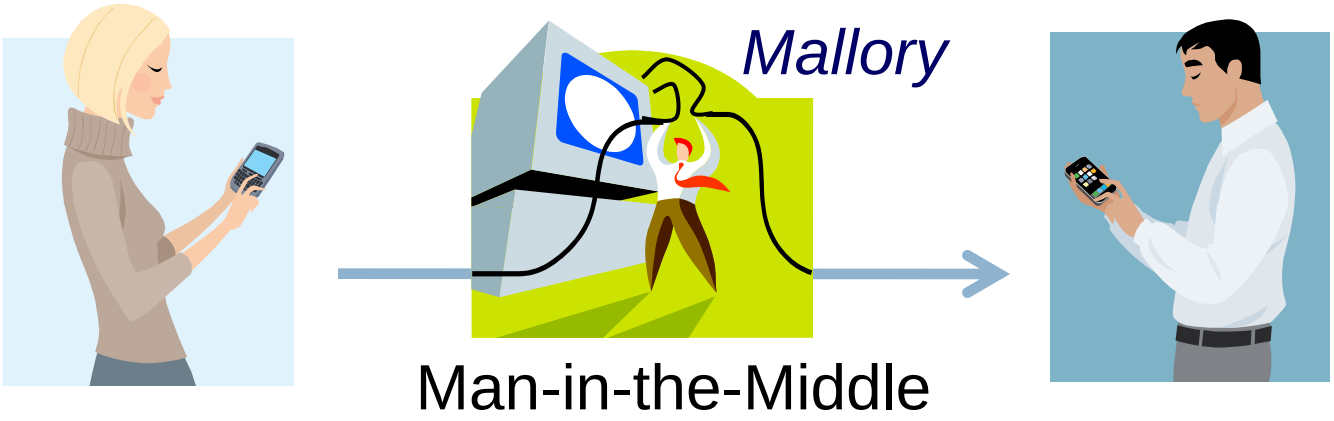
- Symmetric Key Encryption
- Stream and Block Ciphers
- Message-Authentication Codes
- Hash Functions
- Key Establishment
- Asymmetric (Public Key) Encryption

One-slide takeaway

- Extremely tricky to get right

Basic Cryptography Problems

Defends against off-path attacker automatically



Ingredients for a Secure Channel

Confidentiality

Attacker can't see the message

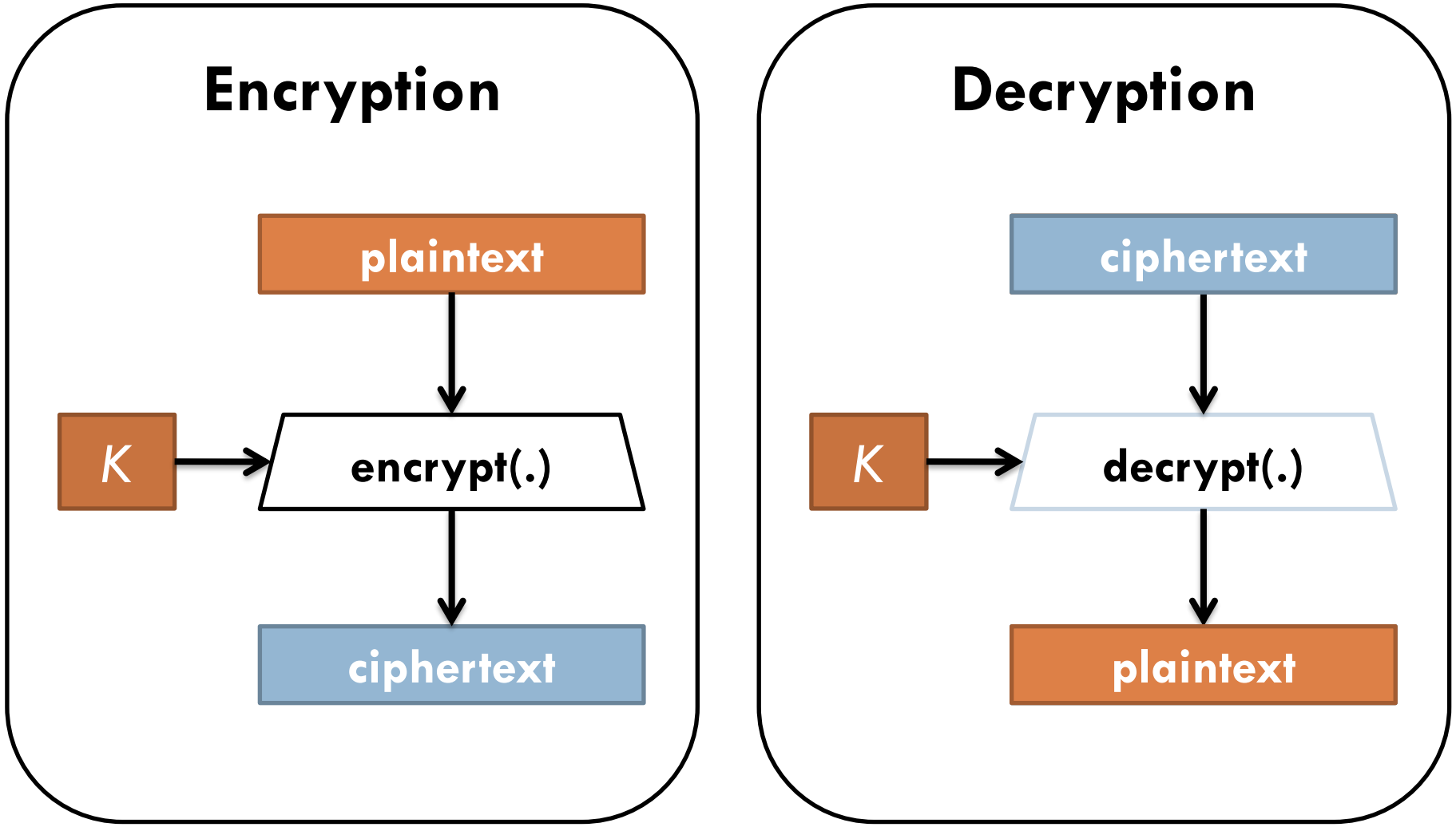
Symmetric Ciphers

Integrity

Attacker can't modify the message

Message Authentication Codes (MACs)

Confidentiality - Symmetric Key Encryption



Substitution cipher

- K = A -> T

  B -> X

  C -> S

  …

  Z -> D

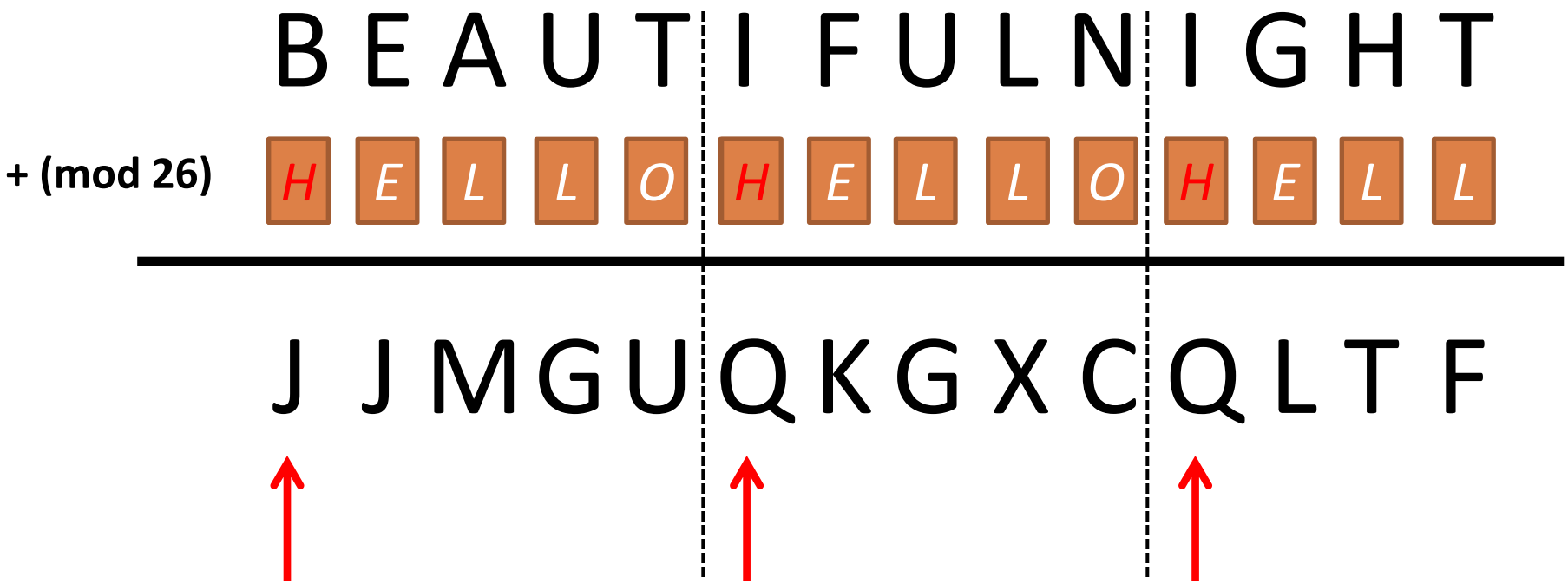
E (K, “ABCZ”) = “TXSD”

D (K, “TXSD”) = “ABCZ”

How to break substitution cypher?

- Stats: The most common letter in English
  - “E” ~ 12.7%
  - “T” ~ 9.1%
  - …
- Stats: The most common pairs of letters in English
  - “HE”
  - “AN”
  - …
- Given ciphertext → plaintext

Vigener cipher (16th century, Rome)



Weakness:

Assume “I” is the most common letter → “Q” is the most common in encrypted letters

→ The first letter of key is “Q” – “I” = “H”

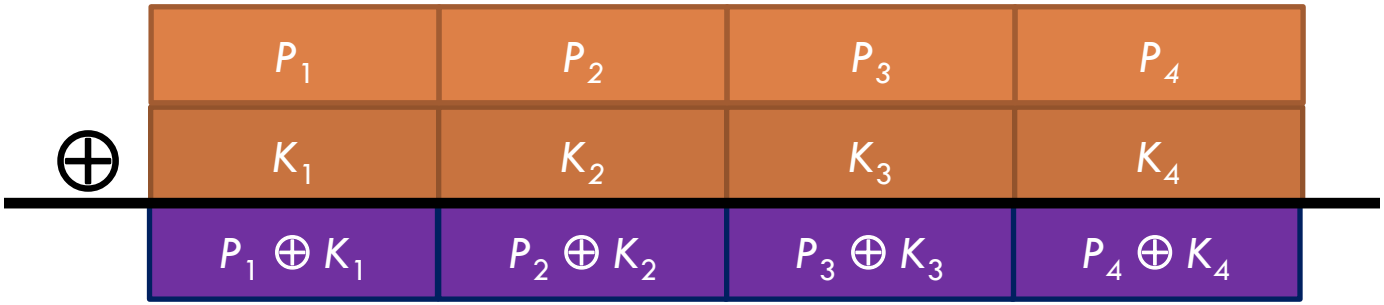
One-Time Pads

First example of a

Provably secure encryption…

Yet may not be practical

One-Time Pads: 1882



$P_i \oplus K_i$	P_i	K_i
0	0	0
0	1	1
1	0	1
1	1	0

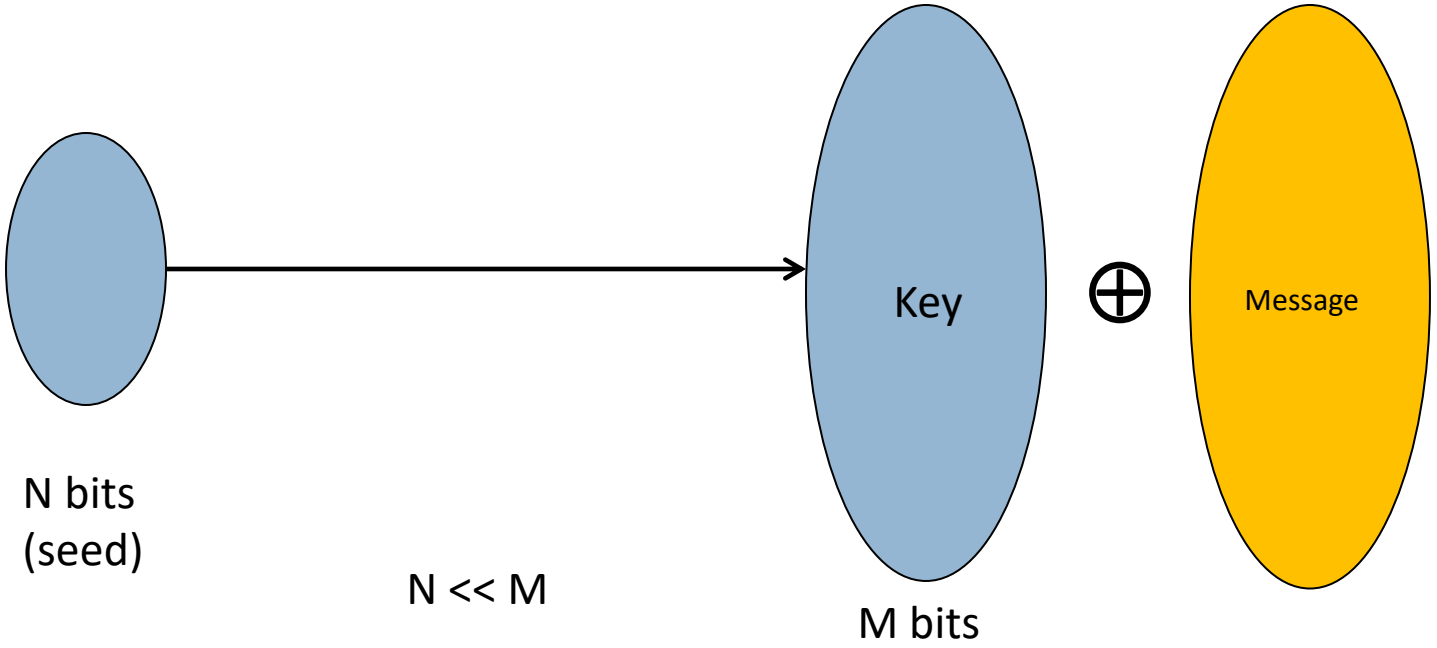
Problems with One-Time Pads

- Long keys (as long as plaintext)
  - Used in special cases in history
  - Not practical
    - Assumes the key safely exchanged
    - Might as well exchange the plaintext itself
- Improvement
  - Stream cipher

Stream cipher: making OTP practical

- Idea: replace "random" key with "pseudorandom" key
  - Exchange once, useful for many messages!

- Pseudo Random Generator (PRG):

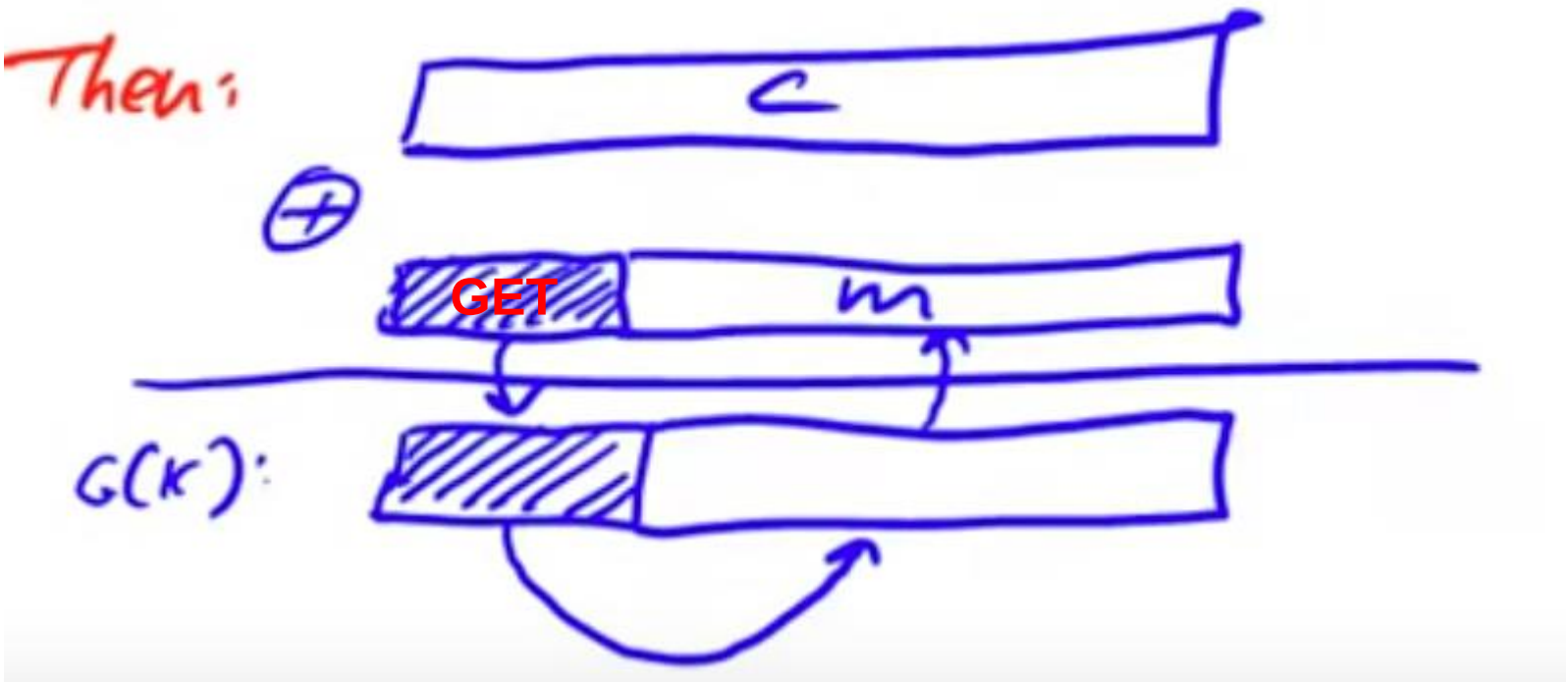


PRG must be unpredictable

- Suppose it is, given earlier bits 1 to i, we can predict bit i+1 to n

- Plaintext attack can reverse the partial key and infer the entire key

An example of HTTP GET request (known plaintext attack)



Weak PRG

- X(n) = a*X(n-1) + b mod c
  - Linear-Congruential Generators
  - Nice properties of statistical randomness, but predictable
  - Fixed total number of states, will repeat at some point

- Glibc random() is also predictable
  - r[i] = (r[i-3] + r[i-31]) % 2^32
  - Output r[i] >> 1

Two-time pad vulnerability

- C1 = M1 $\oplus$ PRG(K)
- C2 = M2 $\oplus$ PRG(K)
- C1 $\oplus$ C2 = ?

Two-time pad vulnerability

- C1 = M1 ⊕ PRG(K)
- C2 = M2 ⊕ PRG(K)
- C1 ⊕ C2 = M1 ⊕ M2

ASCII Code: Character to Binary

0	0011 0000	O	0100 1111	m	0110 1101
1	0011 0001	P	0101 0000	n	0110 1110
2	0011 0010	Q	0101 0001	o	0110 1111
3	0011 0011	R	0101 0010	p	0111 0000
4	0011 0100	S	0101 0011	q	0111 0001
5	0011 0101	T	0101 0100	r	0111 0010
6	0011 0110	U	0101 0101	s	0111 0011
7	0011 0111	V	0101 0110	t	0111 0100
8	0011 1000	W	0101 0111	u	0111 0101
9	0011 1001	X	0101 1000	v	0111 0110
A	0100 0001	Y	0101 1001	w	0111 0111
B	0100 0010	Z	0101 1010	x	0111 1000
C	0100 0011	a	0110 0001	y	0111 1001
D	0100 0100	b	0110 0010	z	0111 1010
E	0100 0101	c	0110 0011	.	0010 1110
F	0100 0110	d	0110 0100	,	0010 0111
G	0100 0111	e	0110 0101	:	0011 1010
H	0100 1000	f	0110 0110	;	0011 1011
I	0100 1001	g	0110 0111	?	0011 1111
J	0100 1010	h	0110 1000	!	0010 0001
K	0100 1011	I	0110 1001	'	0010 1100
L	0100 1100	j	0110 1010	"	0010 0010
M	0100 1101	k	0110 1011	(	0010 1000
N	0100 1110	l	0110 1100	)	0010 1001
				space	0010 0000

Two-time pad vulnerability

- C1 = M1 ⊕ PRG(K)
- C2 = M2 ⊕ PRG(K)
- C1 ⊕ C2 = M1 ⊕ M2

- English encoded in ASCII allows

M1 ⊕ M2 → M1, M2 (recovery fairly easy)

" " = 0100000

"A" – "Z" = 10XXXXX

"A" ⊕ "B" = 00000011

"A" ⊕ "C" = 00000010

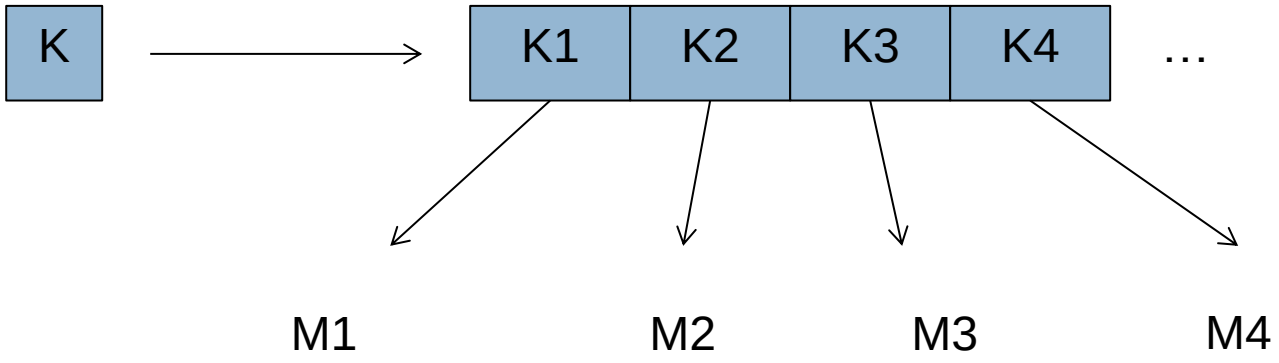
Two-time pad vulnerability

- Never re-use stream cipher key more than once!
  - Example failure: MS-PPTP using the same key bytes twice (in two directions)
  - Another example: 802.11b WEP (PRG seed recycles every 16M frames)

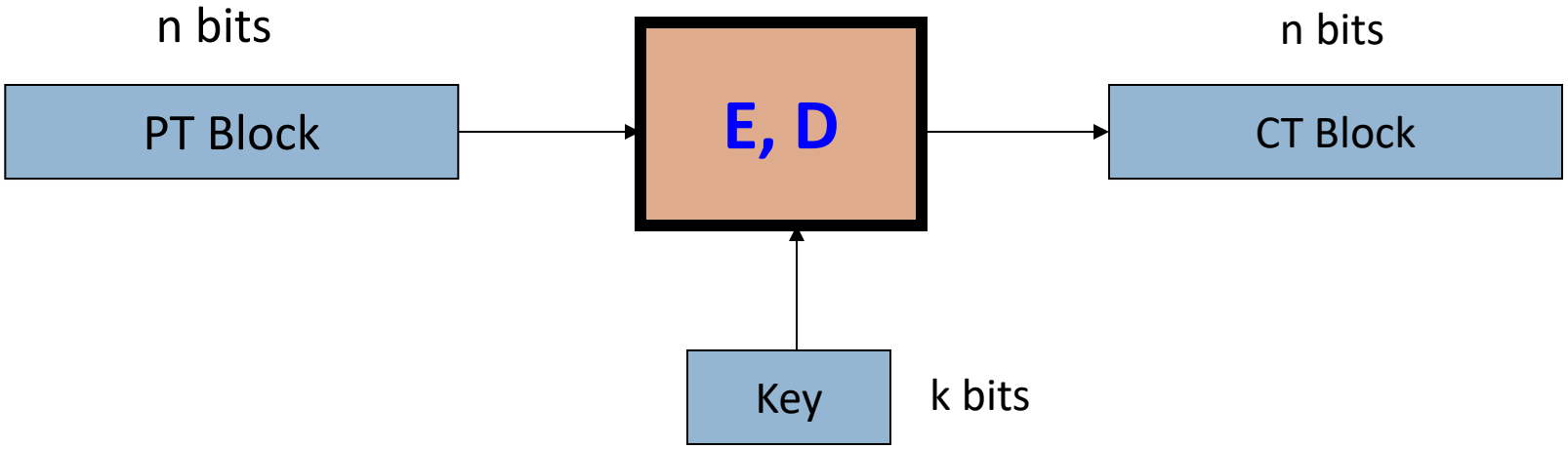
- Session key (e.g., SSL and TLS)
  - Negotiate new key every time, one for client->server, one for server->client.

Never reuse the key

- If you truly want to reuse the same seed (but different keys) to generate multiple messages
  - One way: use PRG(i || K) for ith message
  - A better way



Block ciphers

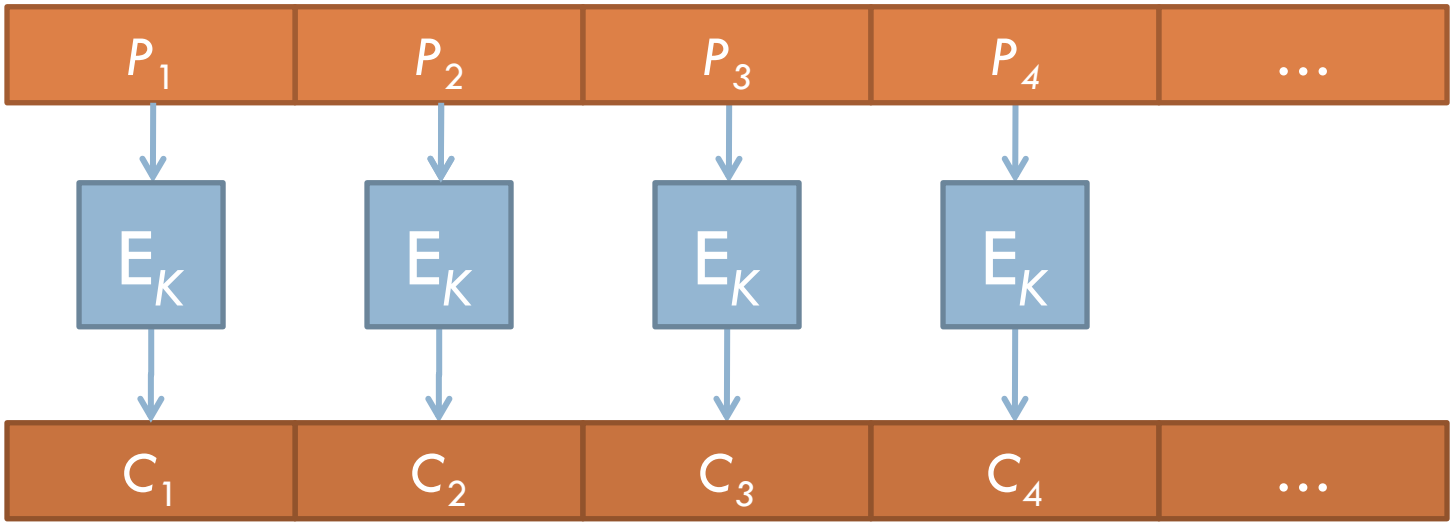


Canonical examples:

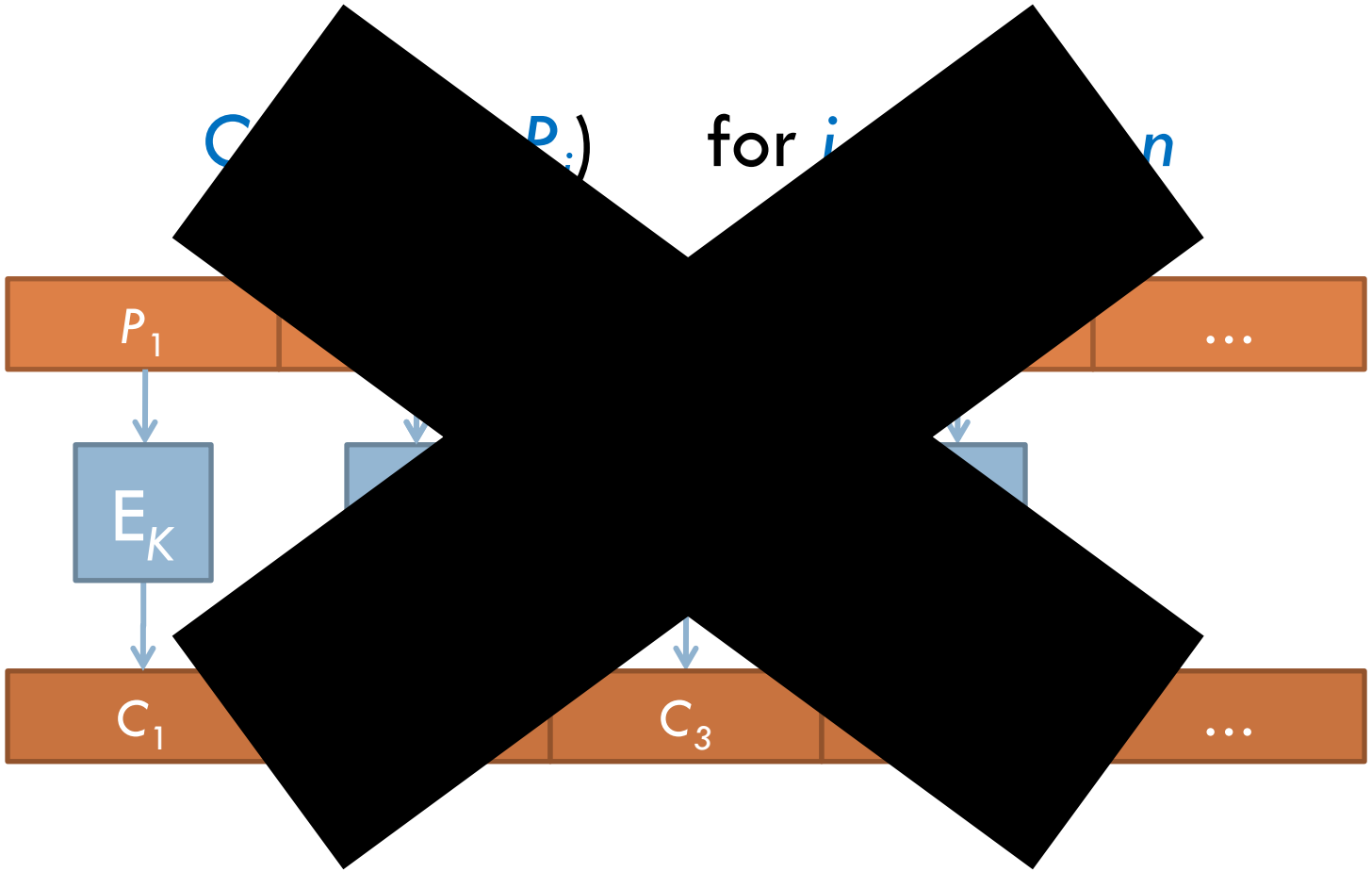
1. 3DES: n= 64 bits, k = 168 bits
2. AES: n=128 bits, k = 128, 192, 256 bits

ECB – Electronic Codebook Mode

$C_i := E(K, P_i)$ for $i = 1, \ldots, n$

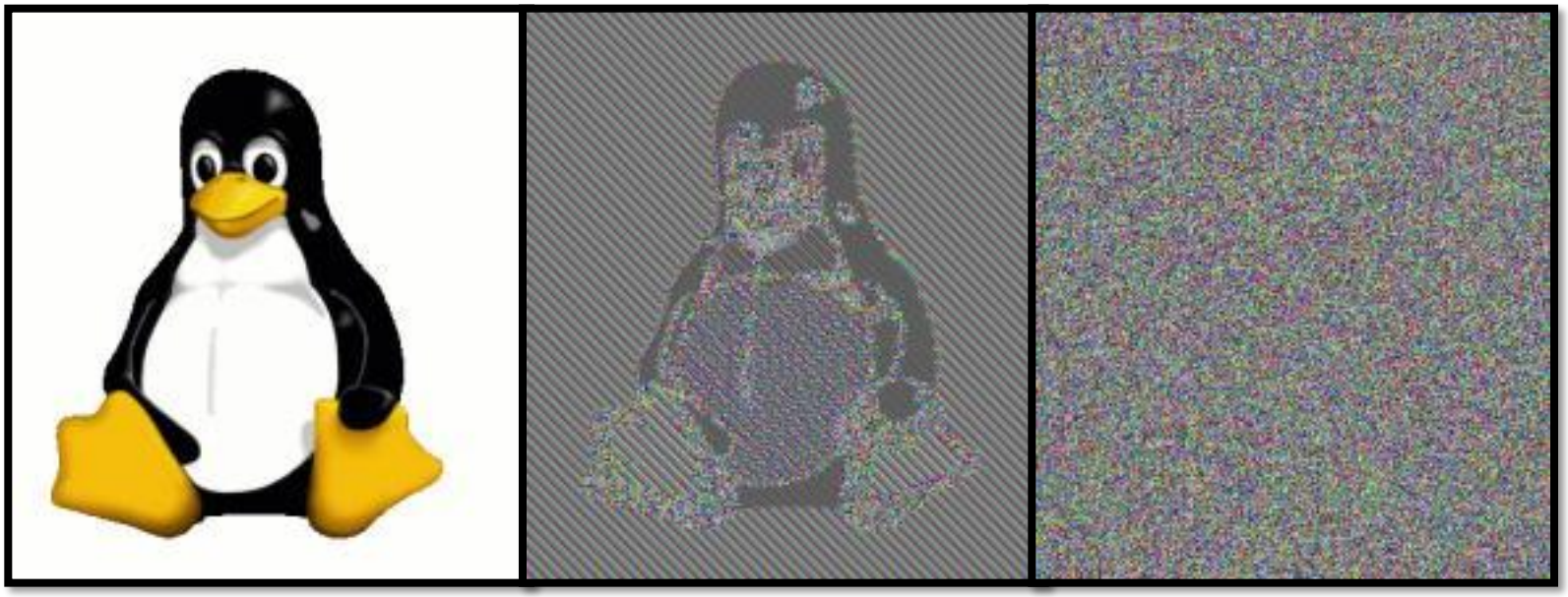


ECB – Electronic Codebook Mode



Why not ECB?

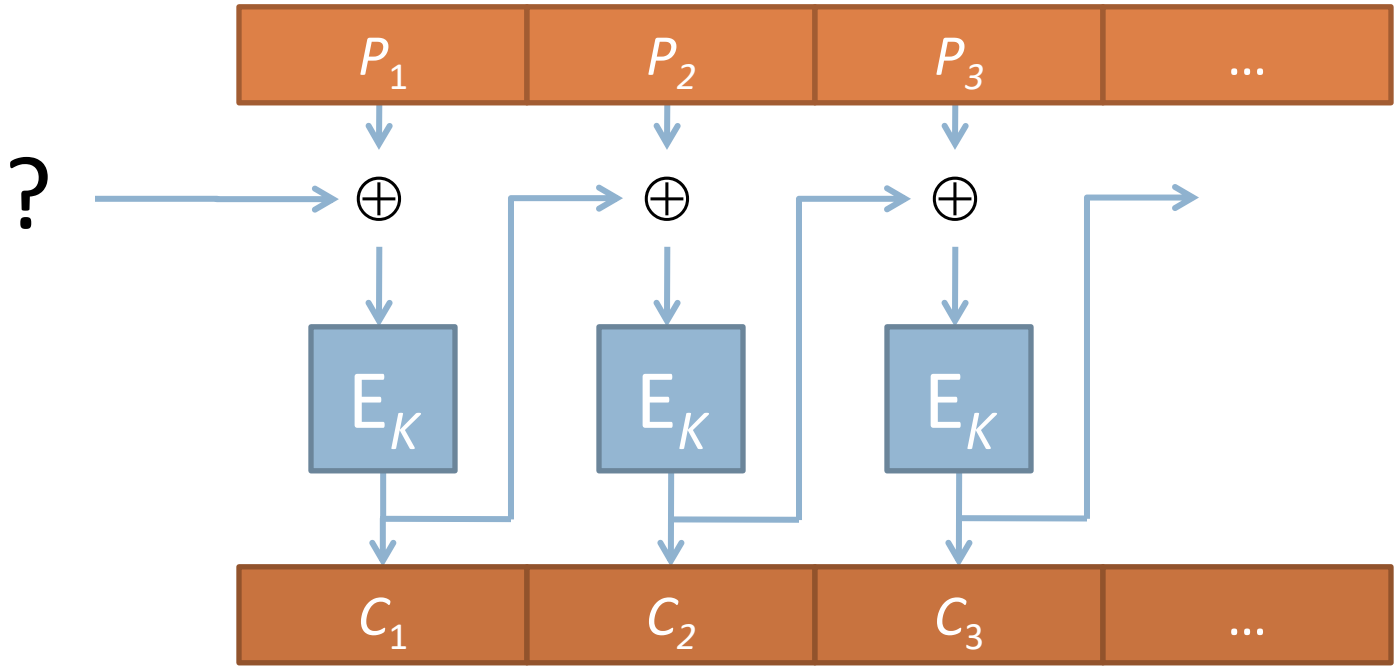
- The cipher text of an identical block is always identical… consider a bitmap image…



(plaintext) (ECB mode) (CBC mode)

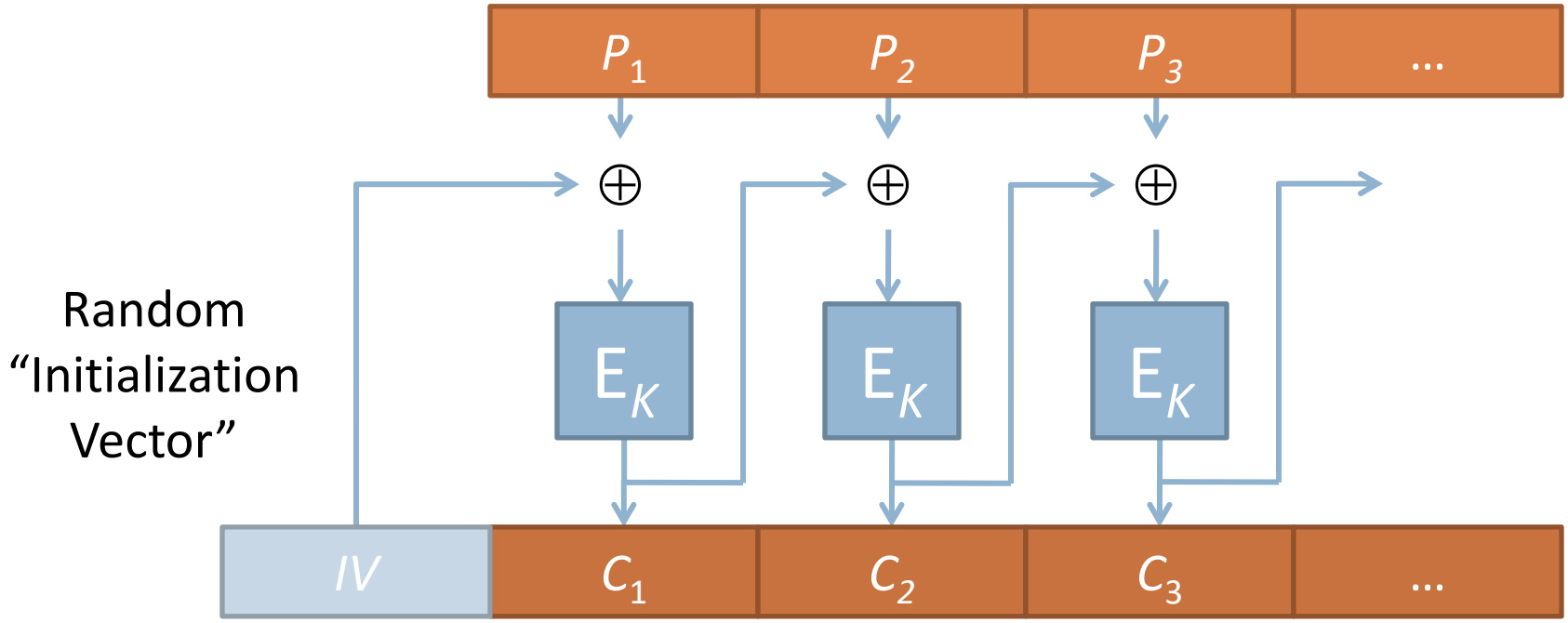
CBC: Cipher-Block Chaining Mode

$C_i := E(K, P_i \oplus C_{i-1})$ for $i = 1, \ldots, n$



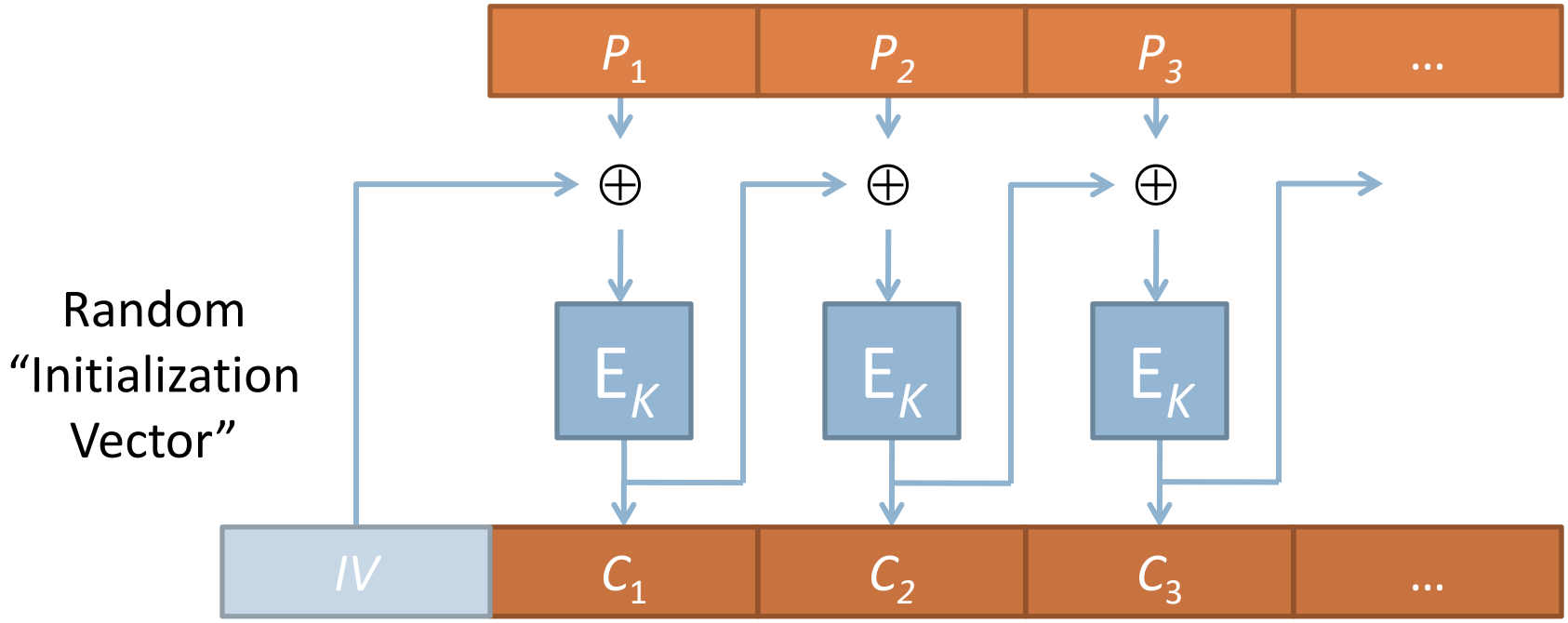
CBC: Cipher-Block Chaining Mode

$C_i := E(K, P_i \oplus C_{i-1})$ for $i = 1, \ldots, n$



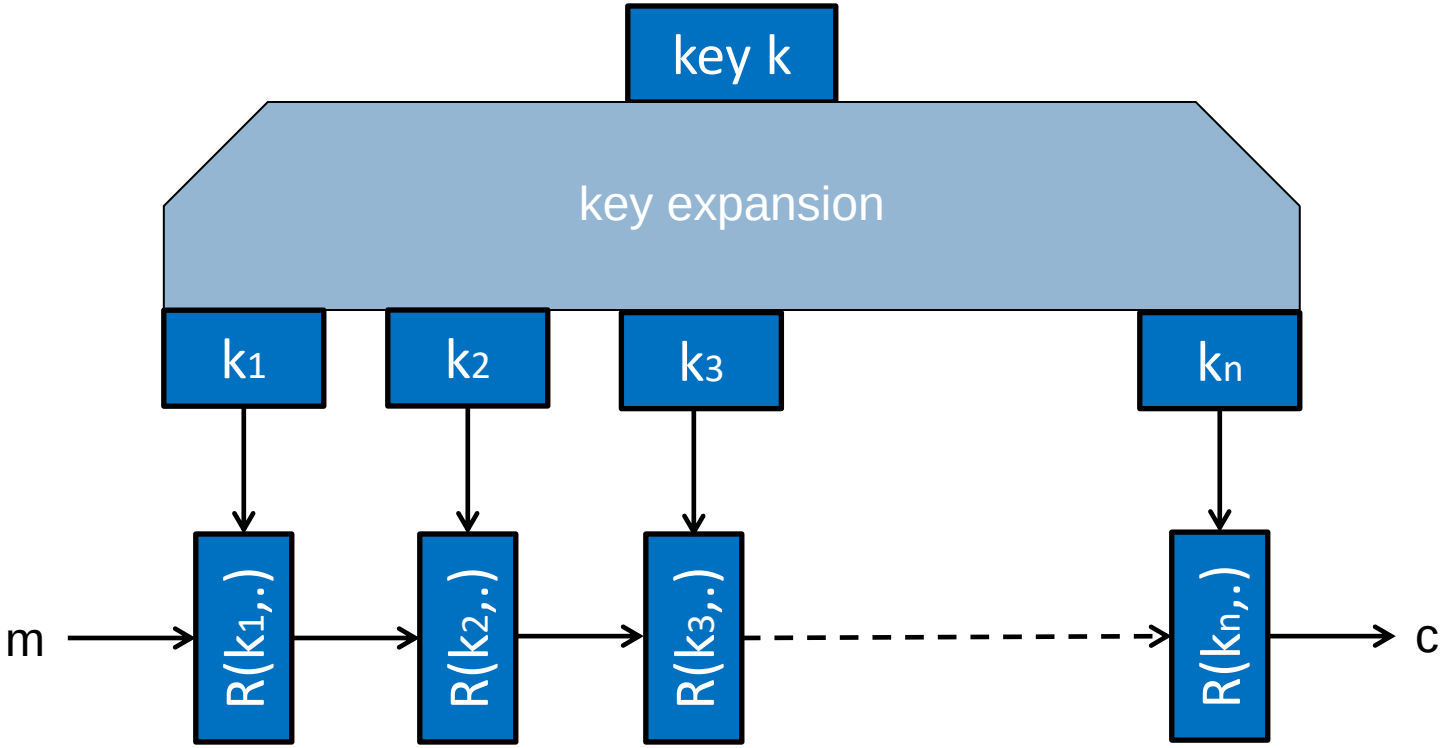
CBC: Cipher-Block Chaining Mode

$C_i := E(K, P_i \oplus C_{i-1})$ for $i = 1, \ldots, n$

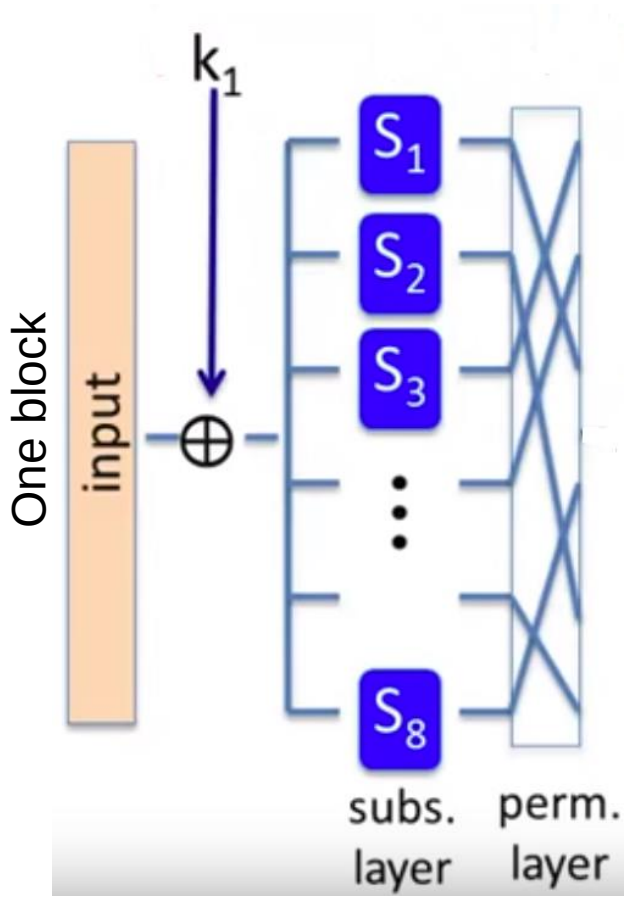


DO NOT REUSE INITIALIZATION VECTORS!!

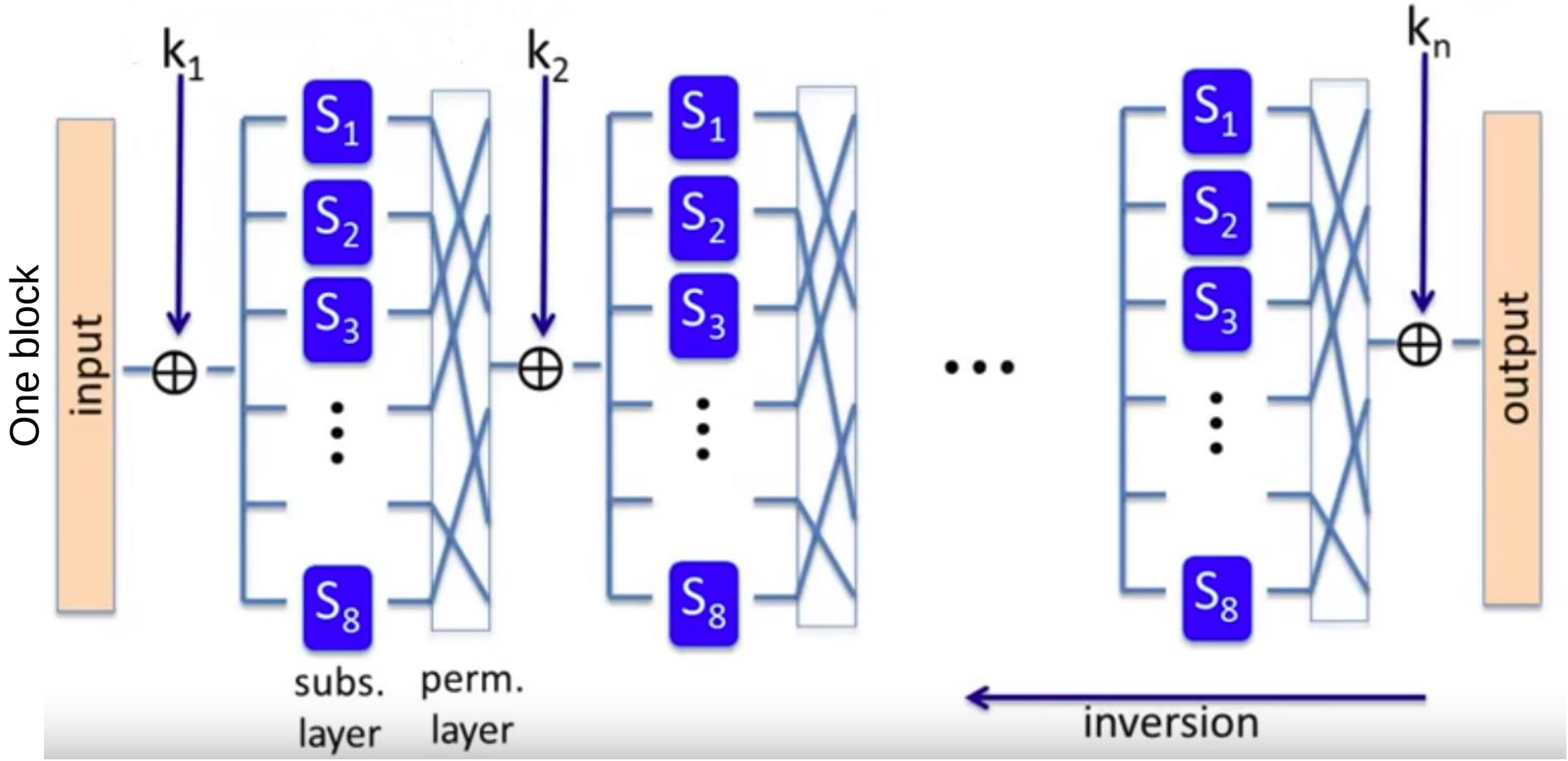
Block Ciphers Built by Iteration



AES: state-of-the-art (2000)



AES: state-of-the-art (2000)



How Safe is AES?

- Known attacks against 128-bit AES if reduced to 7 rounds (instead of 10)
- 128-bit AES very widely used,
  though NSA requires 192- or 256-bit keys for SECRET and TOP SECRET data
- CPU cache timing side channel attacks
  - Lookup table access?
- What should you use?
  - Conservative answer: Use 256-bit AES

So far, confidentiality only, but no integrity

- $c = m \oplus k$
- Attacker can change the ciphertext by

  $c \oplus p$
- When decrypted,

  $(c \oplus p) \oplus k = ?$

  $(c \oplus p) \oplus k = ((m \oplus k) \oplus p) \oplus k =$ 
- Assuming m has a single bit.
  - 1 indicates attack
  - 0 indicates retreat
  - Can always force the wrong decision (not knowing what m is)

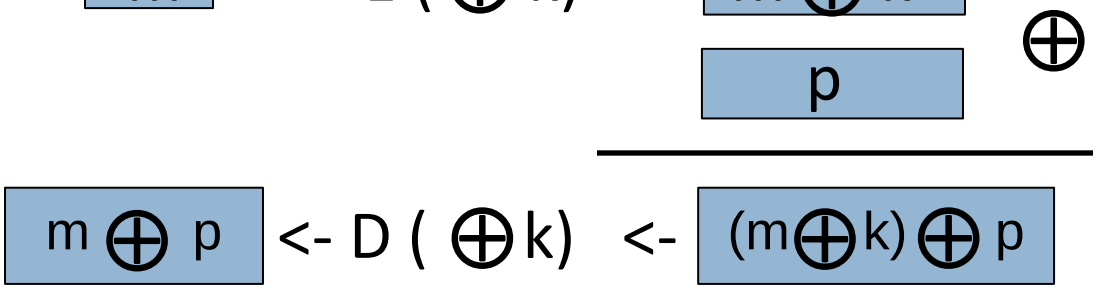
Integrity

- Goal: **Integrity**, independent of confidentiality

- Example:
  - Protecting public binaries on disk
  - Protecting ads on web pages

Attack against integrity

- OTP is vulnerable (malleable)
  - m -> E (⊕ k) -> m ⊕ k



Modifications are undetectable

And have predictable impact

Attack against integrity (known plaintext)

- “From Bob…” -> E=“From Bob…” ⊕ k

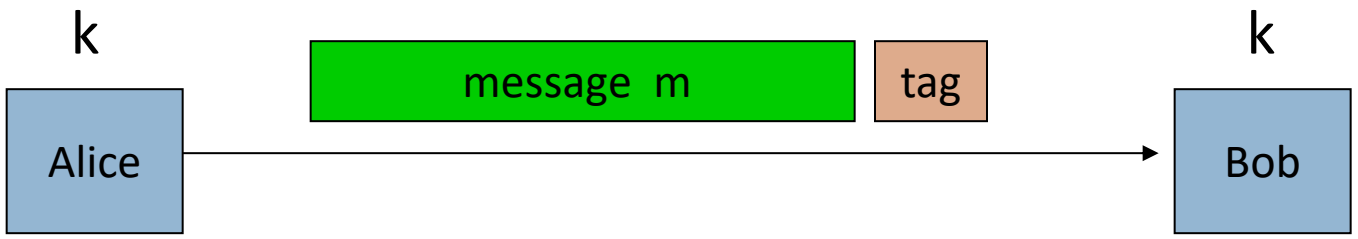
???

⊕

“From Eve”

Bob ⊕ Eve = 0x07 0x19 0x07

Integrity: Message Authentication Code (MAC)



Generate tag:

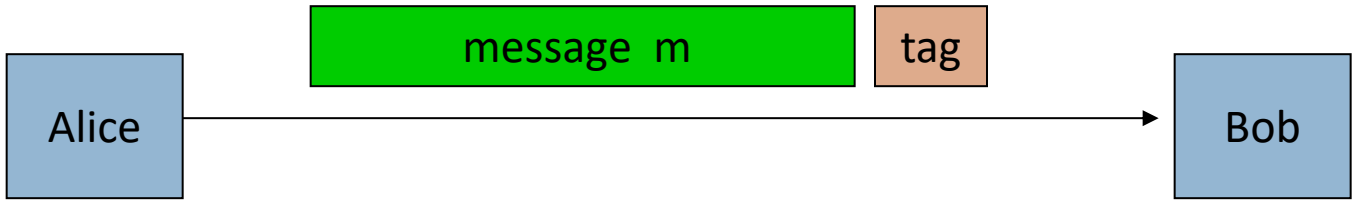
$tag \leftarrow S(k, m)$

Verify tag:

$V(k, m, tag) \stackrel{?}{=}$ 'yes'

Typical solution is to compute a hash function as the tag = H(m, k)

Integrity requires a secret key



Generate tag:
tag ← CRC(m)

Verify tag:
V(m, tag) $\stackrel{?}{=}$ `yes'

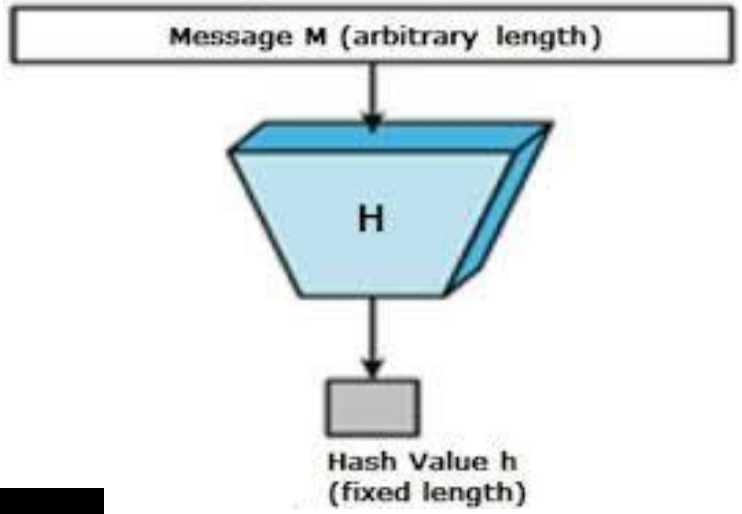
Is it safe to use a checksum function?

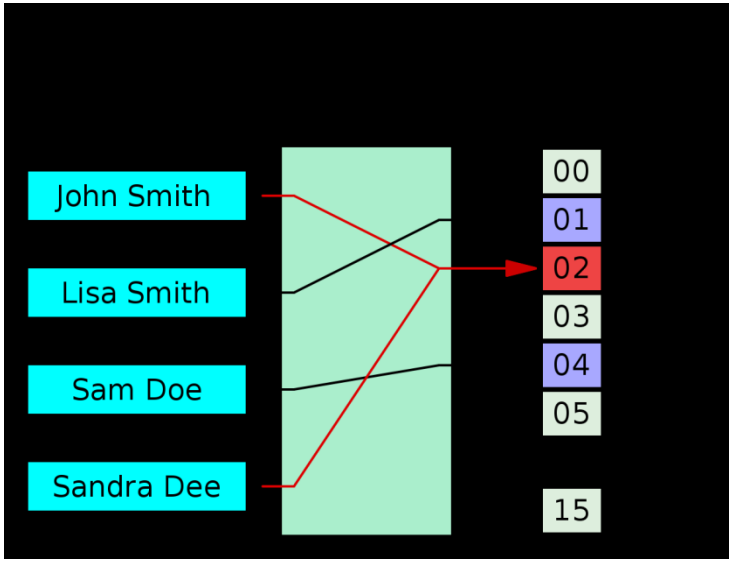
- Attacker can easily modify message m and re-compute CRC.

- CRC designed to detect **random**, not malicious errors.

Hash Functions

- Ideal: Random mapping from *any arbitrary-size input* to a *fixed size output*



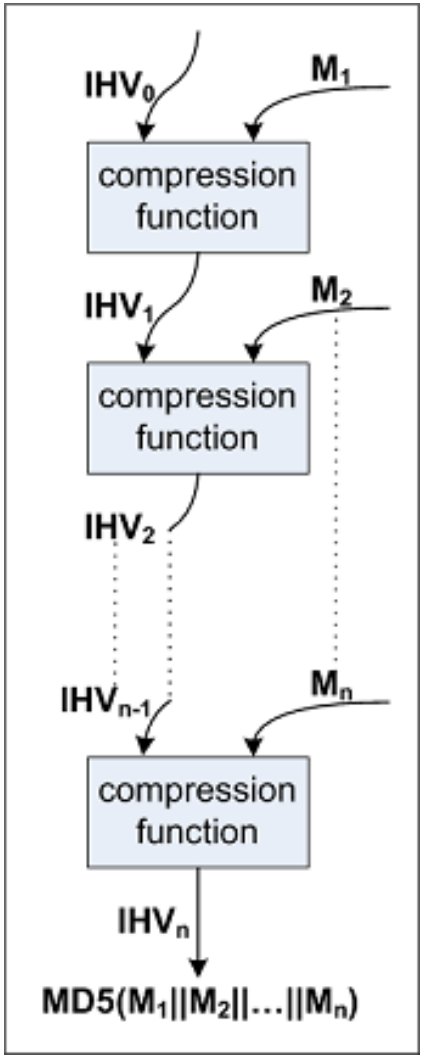


Hash Function Requirements

- First pre-image
  - Given h(x), find x
- Second pre-image
  - Given m_1, find m_2 s.t. $h(m_1) = h(m_2)$
- Collision
  - Given nothing, find *any* m_1 != m_2 s.t. $h(m_1) = h(m_2)$
  - Birthday Attack: 70% for 30 students

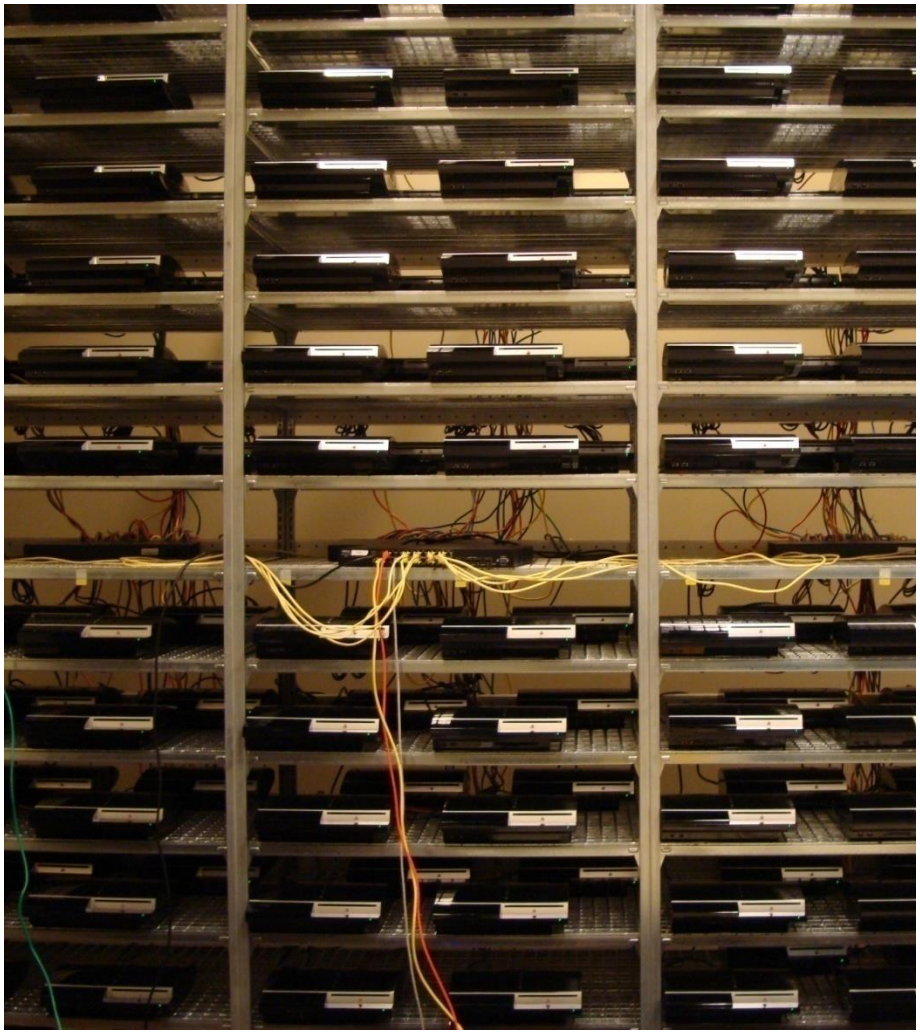
MD5 Hash Function

- Designed in 1992 by Ron Rivest
  - 128-bit output
  - 128-bit internal state
  - 512-bit block size
- Like most hash functions, uses block-chaining construction



MD5 is Unsafe – Never use it!

- First flaws in 1996;
  by 2007, researchers demonstrated a collision
- Dec. 2008:
  others used this to fake SSL certificates (cluster of 200 PS3s)



MD5 Collision


```
d131dd02c5e6eec4693d9a0698aff95c 2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e48883257 1415a 085125e8f7cdc99fd91dbdf280373c5b
d8823e3156348f5bae6dacd436c919c6 dd53e2b487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080a80d1e c69821bcb6a8839396f9652b6ff72a70

d131dd02c5e6eec4693d9a0698aff95c 2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325f1415a 085125e8f7cdc99fd91dbd7280373c5b
d8823e3156348f5bae6dacd436c919c6 dd53e23487da03fd02396306d248cda0
e99f33420f577ee8ce54b6708280d1e c69821bcb6a8839396f965ab6ff72a70
```

Both of these blocks hash to 79054025255fb1a26e4bc422aef54eb4

SHA Hash Functions

- SHA-1 – standardized by NIST in 1995
  - 160-bit output and internal state
  - 512-bit block size
- SHA-2 – extension published in 2001
  - 256 (or 512)-bit output and internal state
  - 512 (or 1024)-bit block size
- SHA-3 – chosen by NIST in 2012
  - 256 (512)-bit output
  - Different “sponge” construction

Construction of MAC

Assuming we have a good hash function H:

Basic idea: Key is hashed together with the message

MAC1(K,m) = H (K || m)?

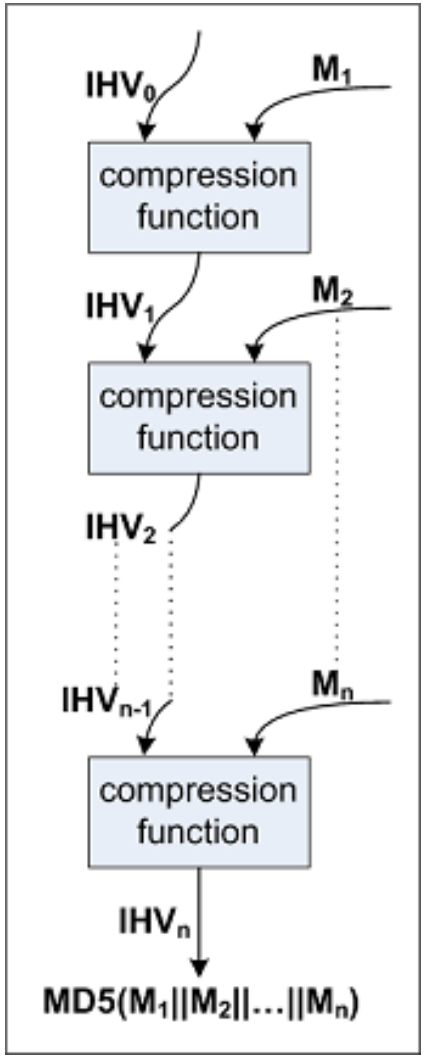
MAC2(K,m) = H (H (m) || K)?

Vulnerable MAC construction?

- $H(K||m) = H(K||m_1, m_2, \ldots, m_n) = MAC$
- Length extension attack

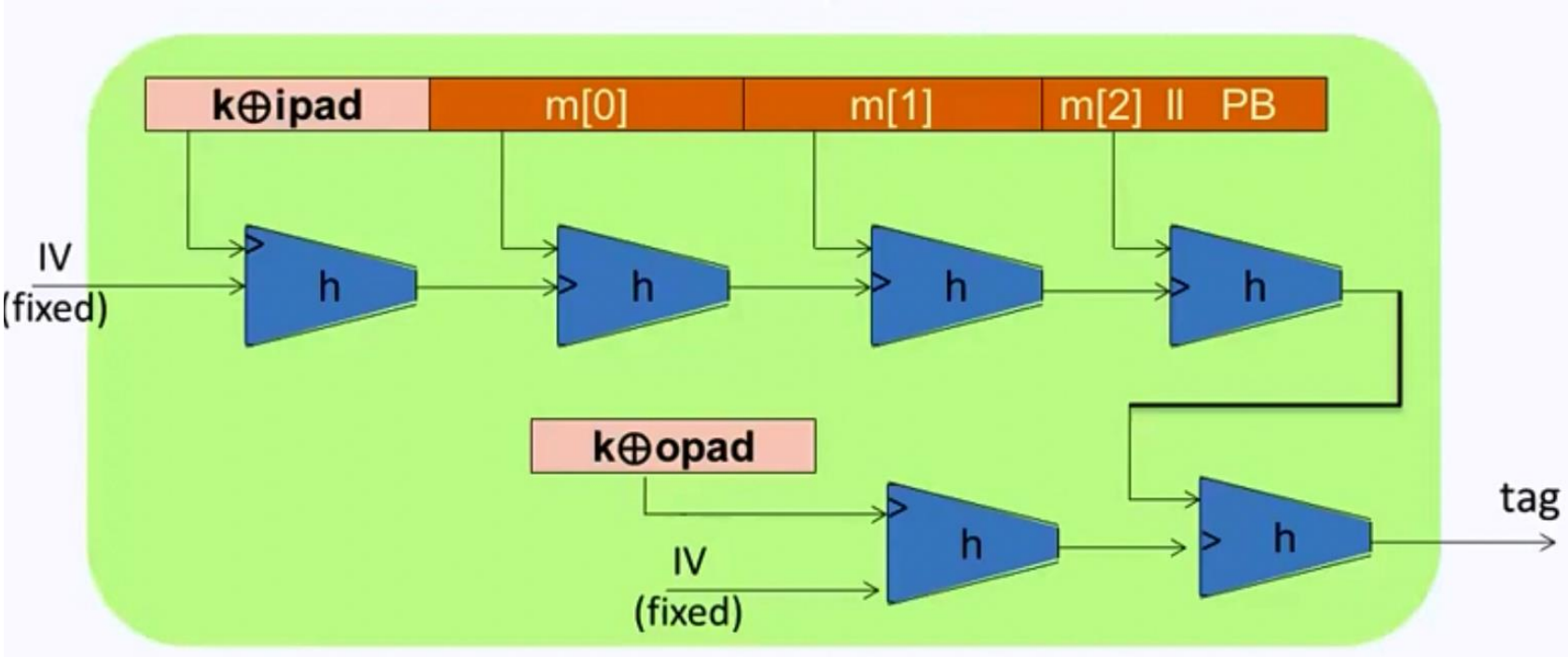
Length Extension Attack

- Block-Chaining Construction
  - Also known as Merkle-Damgaard hash
- Vulnerable to length extension attack
- MAC = H (K||m_1||..m_n)
- Modified m and MAC
  - MAC_A = H (MAC || m_{n+1})
  - m_A = m_1||..m_n|| m_{n+1}



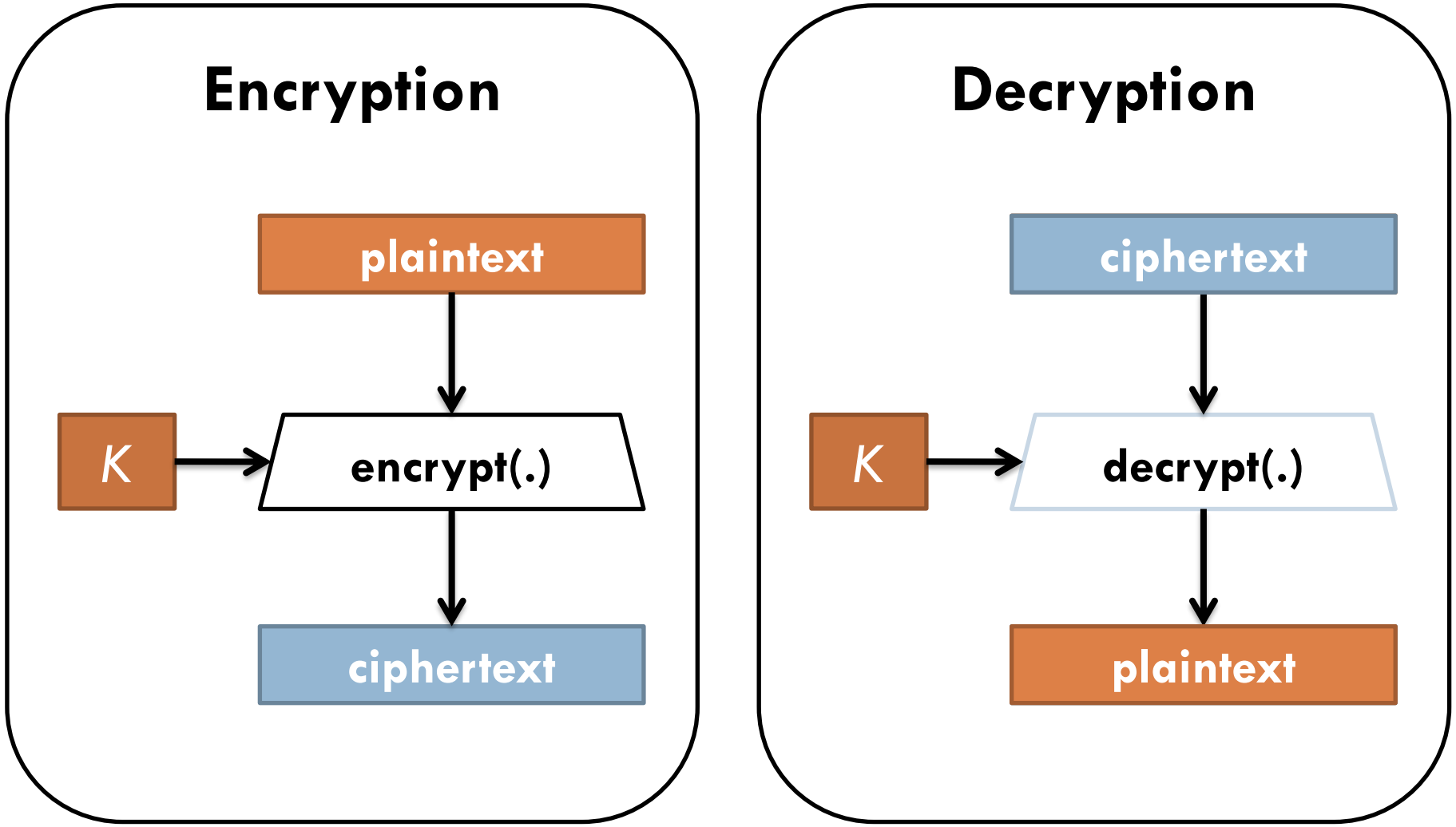
HMAC is not vulnerable

- HMAC(K,m) = H(($K \oplus$ opad) || H($K \oplus$ ipad || m))



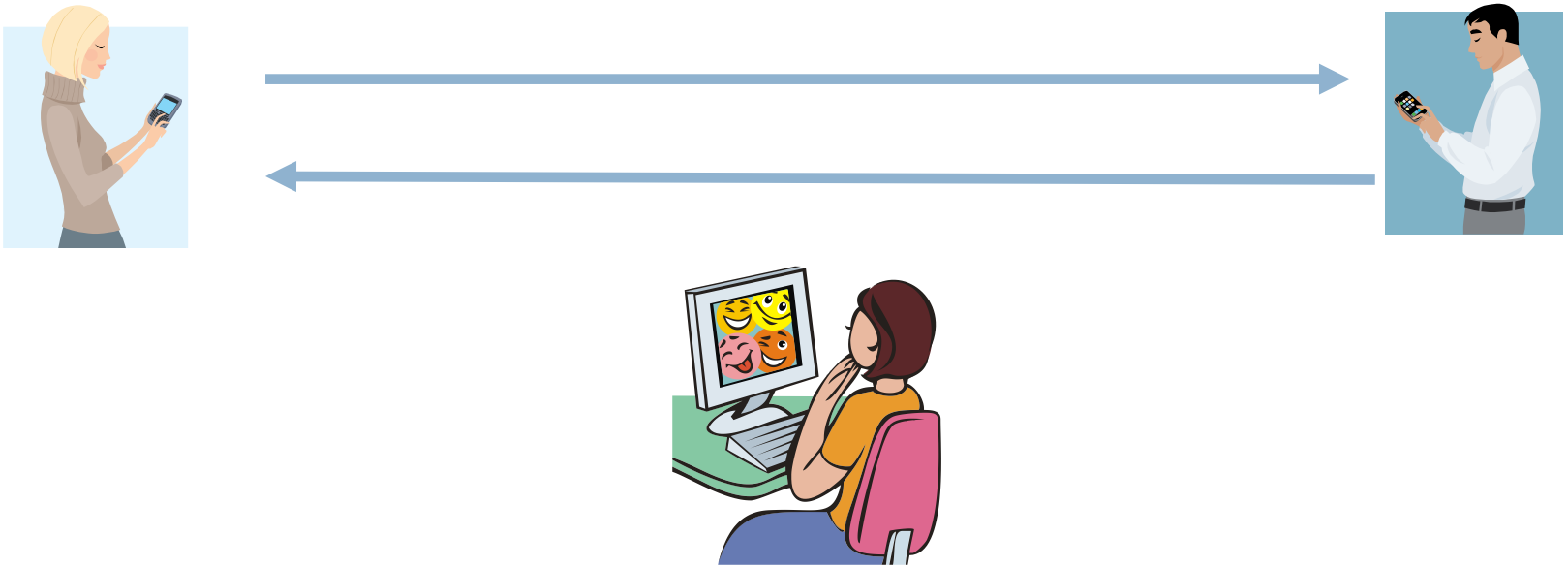
HMAC provides nice provable security properties

Symmetric Key Encryption



Key establishment: Chicken-and-egg problem

- How do we share our symmetric key in front of an eavesdropping adversary?
  - Can this be done with an exponential gap?



Eavesdropper??

Public Key Cryptography

- Symmetric key cryptographic is great… but has the fundamental problem that every send-receiver pair must establish a secret out of nothing …
  - Hard to bootstrap!
  - Can we rely on some public info that everyone agrees on?
- Also known as “Asymmetric Encryption”

Diffie-Hellman Key Exchange

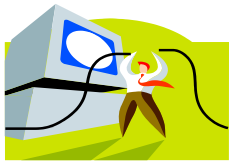
- "Key Exchange" developed by Whitfield Diffie and Martin Hellman in 1976

- Based on *Discrete Log Problem* which we believe is difficult ("the assumption")
  - "One-way" functions
  - $b^k = a \rightarrow$ finding the integer k is difficult
    - $k = \log_b a$

Diffie-Hellman Key Exchange

1. Alice generates and shares ***g*** with Bob
2. Alice and Bob each generate a secret number, which we denote ***x*** and ***y*** ("private key")
3. Alice generates g^x and sends it to Bob ("public key")
4. Bob generates g^y and sends it to Alice ("public key")
5. Alice calculates $(g^y)^x$ and Bob calculates $(g^x)^y$
6. Alice and Bob have $(g^y)^x = g^{xy} = g^{yx} = (g^x)^y$

Attacking Diffie-Hellman (MITM)



Mallory

Chooses random x

g^x

Chooses random v

g^v

Chooses random y

g^y

Chooses random w

g^w

$k := (g^w)^x$

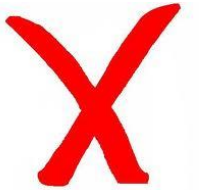
$k := (g^w)^x$
$k' := (g^v)^y$

$k' := (g^v)^y$

Summary of Goals

Confidentiality

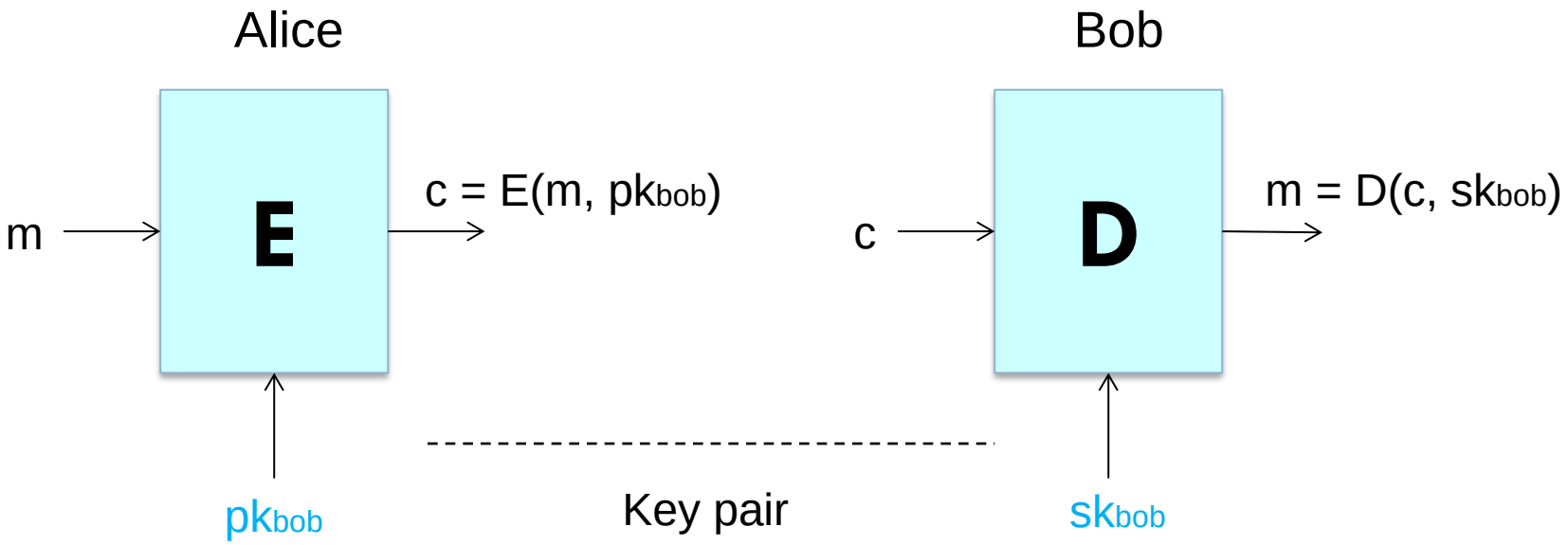
Integrity



Authentication

Public Key Cryptography

- Two keys
  - *Private key* (sk) known only to individual
  - *Public key* (pk) available to anyone



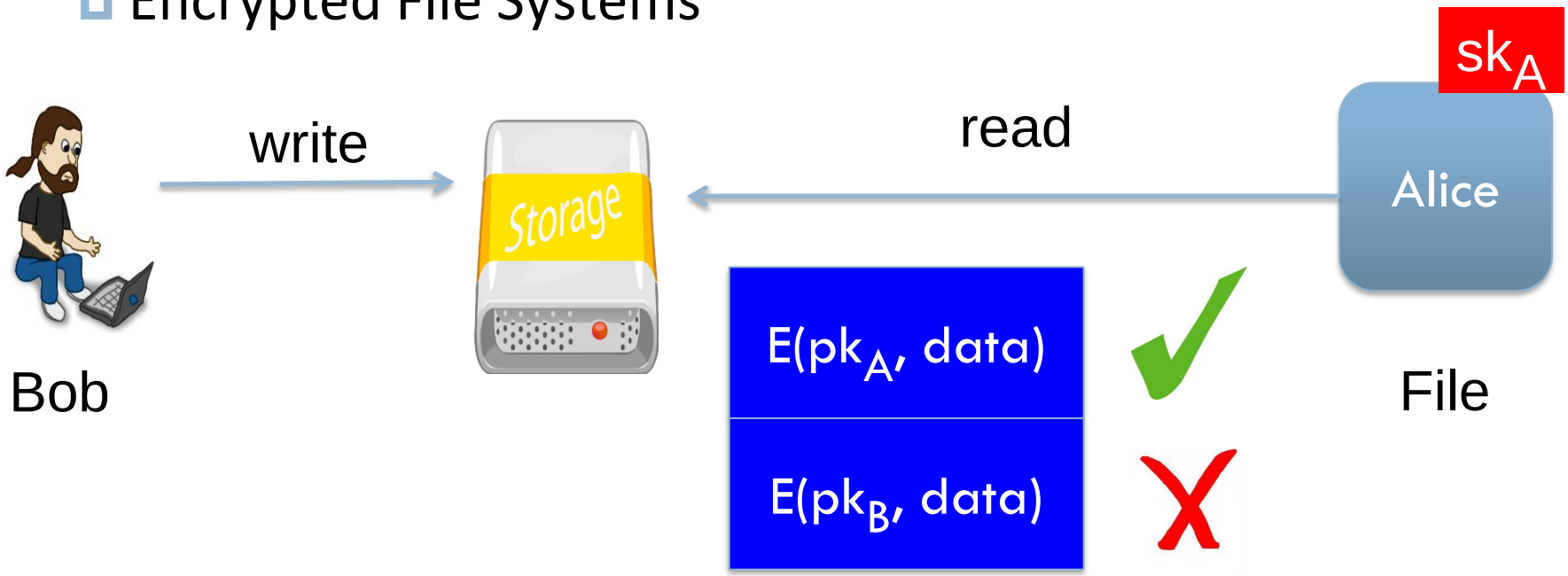
Public Key Encryption

Def: a public-key encryption system is a triple of algs. (G, E, D)

- G(): randomized alg. outputs a key pair (pk, sk)
- E(m, pk): alg. that takes $m \in M$ and outputs $c \in C$
  - To talk to a person, we use their public key to encrypt
- D(c, sk): alg. that takes $c \in C$ and outputs $m \in M$ or error
  - Only that person can decrypt

Public-Key Encryption Applications

- Non-interactive:
  - Secure Email (PGP): Bob has Alice's pub-key and sends her an email
  - Encrypted File Systems

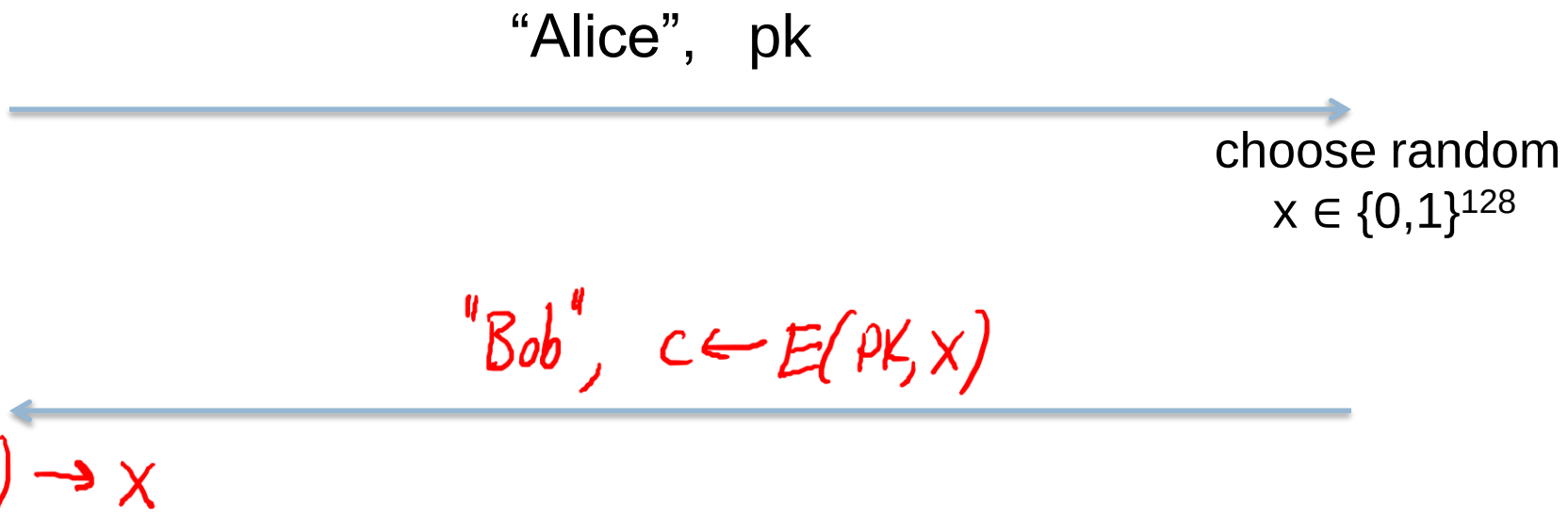


Establishing a Shared Secret – eavesdropping attacker only

Key exchange (e.g., in HTTPS)

Alice **Bob**

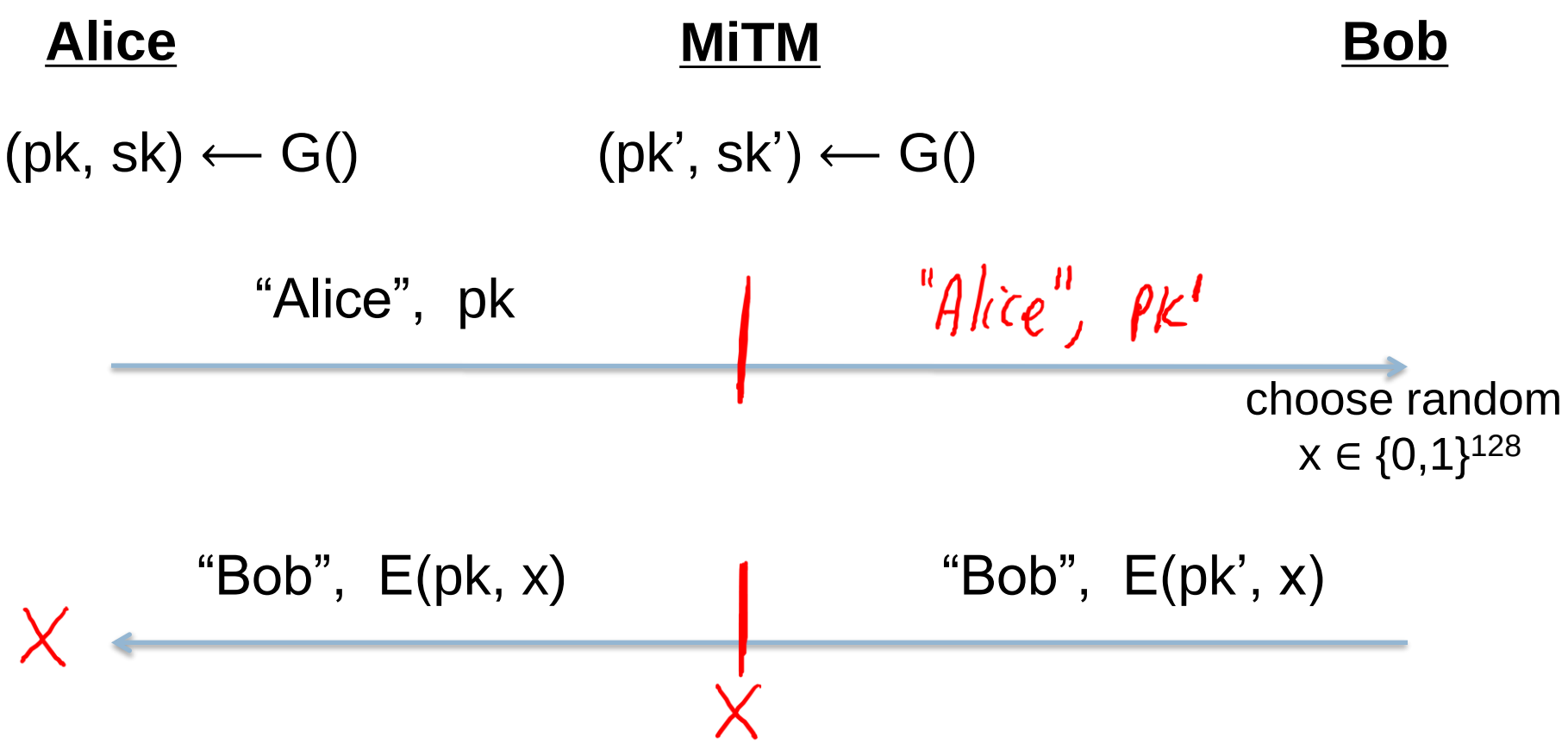
(pk, sk) ⟵ G()



X: shared secret

Insecure Against Man-in-The-Middle

As described, the protocol is insecure against **active** attacks



Public Key Encryption – Integrity/Authenticity

Alice **Bob**

(pk, sk) ⟵ G()

Still a chicken-and-egg problem?

How can Bob obtain Alice's pk?

Establishing Trust

- **How can Bob know what Alice's public key is?**

- Web of Trust (e.g. PGP)
  - Decentralized – everybody can publish their own public key
- Trust on First Use (TOFU) (e.g. SSH)
- Public Key Infrastructure (PKI) (e.g. SSL)
  - Centralized

What is PKI?

- Organizations we trust (often known as "Certificate Authorities") generate certificates to tie a public key to an organization

- We trust that we're talking to the correct organization if we can verify their public key with a trusted authority

In the Hands of Few

- Symantec (which bought VeriSign's SSL interests) with 35.7% market share
- Comodo SSL with 26.9%
- GlobalSign with 14.9%
- Go Daddy with 13.0%
- DigiCert with 3.4%

- Alternative: DANE (DNSSEC-based)

SSL/TLS Certificates

Subject: C=US/O=Google Inc/CN=www.google.com
Issuer: C=US/O=Google Inc/CN=Google Internet Authority
Serial Number: 01:b1:04:17:be:22:48:b4:8e:1e:8b:a0:73:c9:ac:83
Expiration Period: Jul 12 2010 - Jul 19 2012
Public Key Algorithm: rsaEncryption
Public Key:
43:1d:53:2e:09:ef:dc:50:54:0a:fb:9a:f0:fa:14:58:ad:a0:81:b0:3d
7c:be:b1:82:19:b9:7c3:8:04:e9:1e5d:b5:80:af:d4:a0:81:b0:b0:68:5b:a4:a4
:ff:b5:8a:3a:a2:29:e2:6c:7c3:8:04:e9:1e5d:b5:7c3:8:04:e9:39:23:46

Signature Algorithm: sha1WithRSAEncryption

Signature: 39:10:83:2e:09:ef:ac:50:04:0a:fb:9a:f0:fa:14:58:ad:a0:81:b0:3d
7c:be:b1:82:19:b9:7c3:8:04:e9:1e5d:b5:80:af:d4:a0:81:b0:b0:68:5b:a4:a4
:ff:b5:8a:3a:a2:29:e2:6c:7c3:8:04:e9:1e5d:b5:7c3:8:04:e9:1e:5d:b5

Signatures on Certificates

- c = E(m, sk) // only owner can encrypt
- m = D(c, pk) // anyone can decrypt
- Mathematical properties (e.g., RSA)

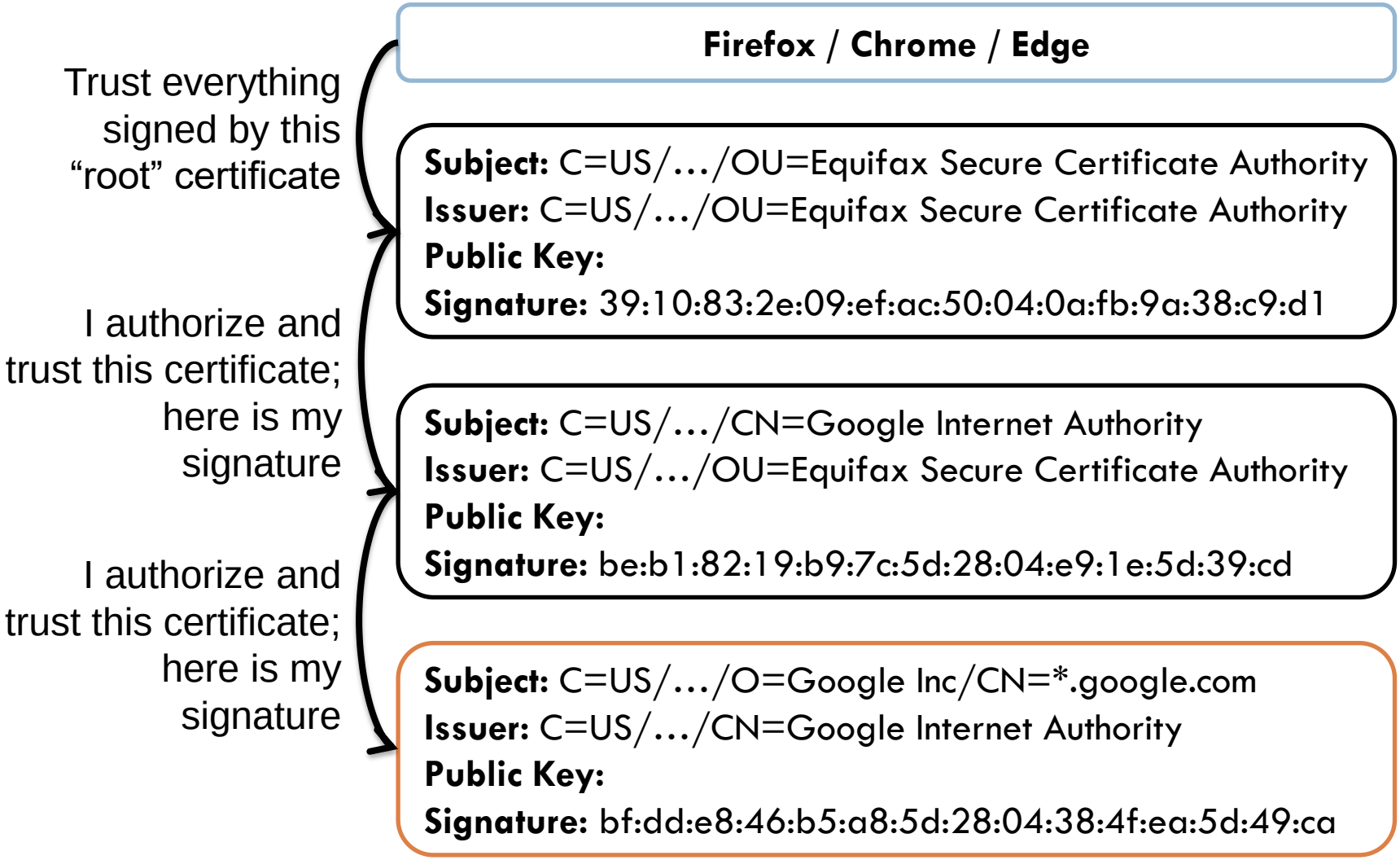
- signature = E(sk, certificate)
- <signature, certificate> can be validated by anyone (but too long)

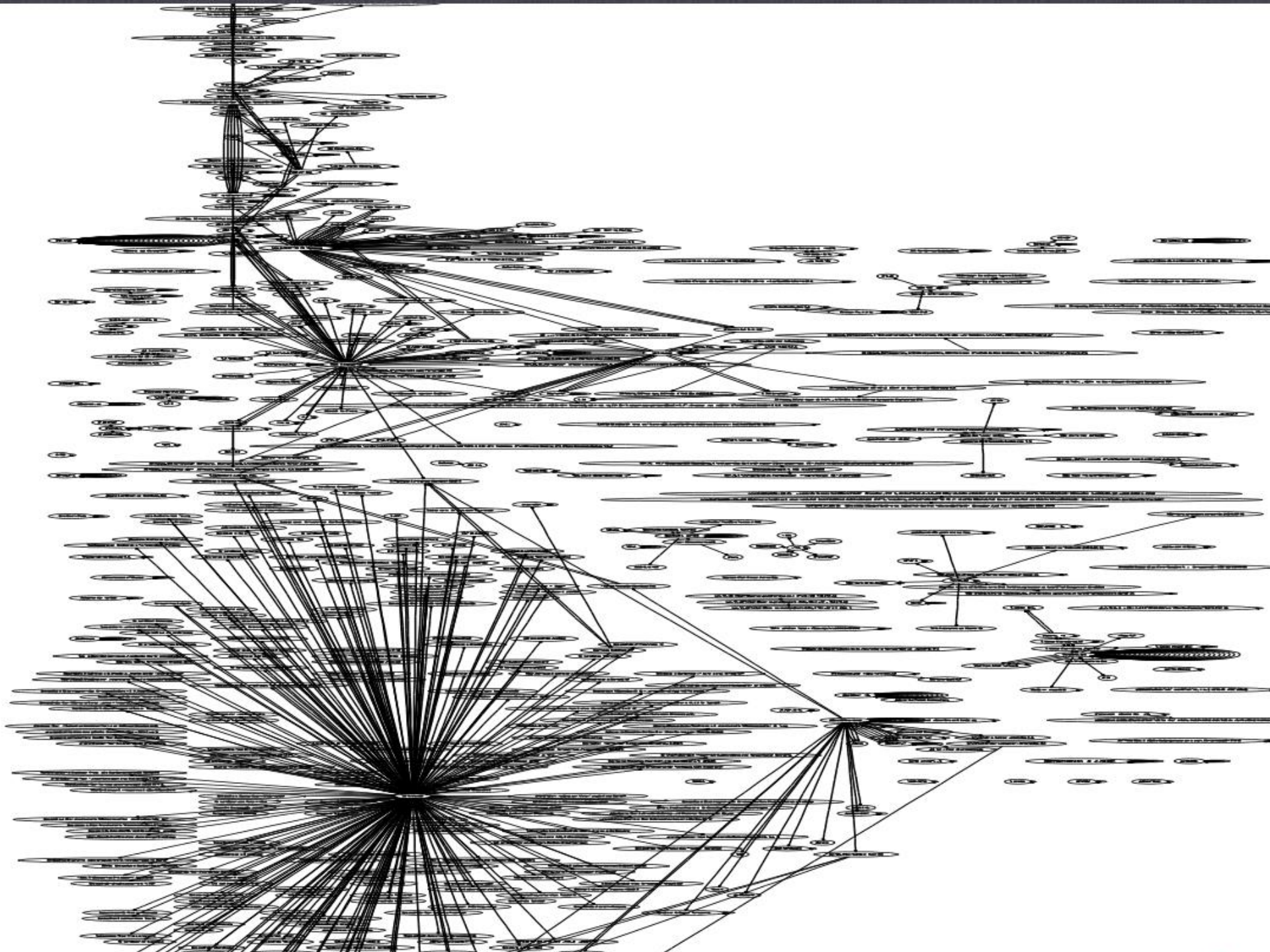
- Oftentimes see a signature algorithm such as **sha1WithRSAEncryption**

- **Sigature = Encrypt_{SK}(SHA-1(certificate))**

- **SHA-1(certificate) = Decrypt_{PK}(signature)**

Certificate Chains



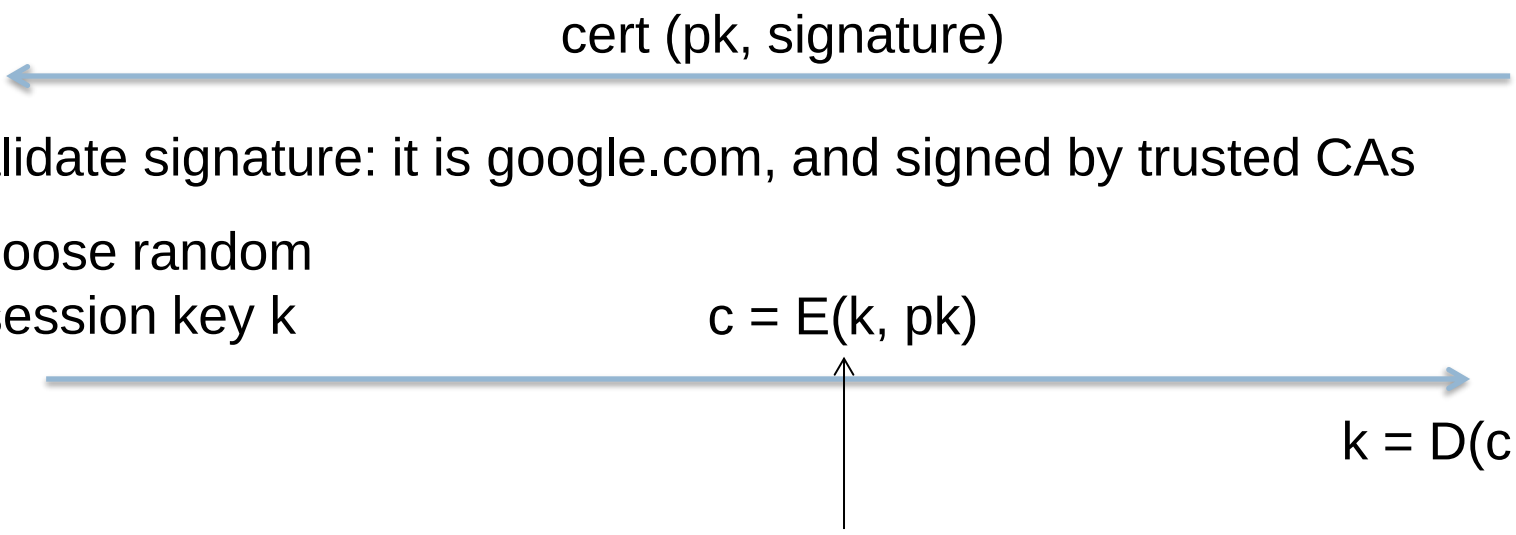


Public Key Cryptography -- putting everything together – SSL/TLS

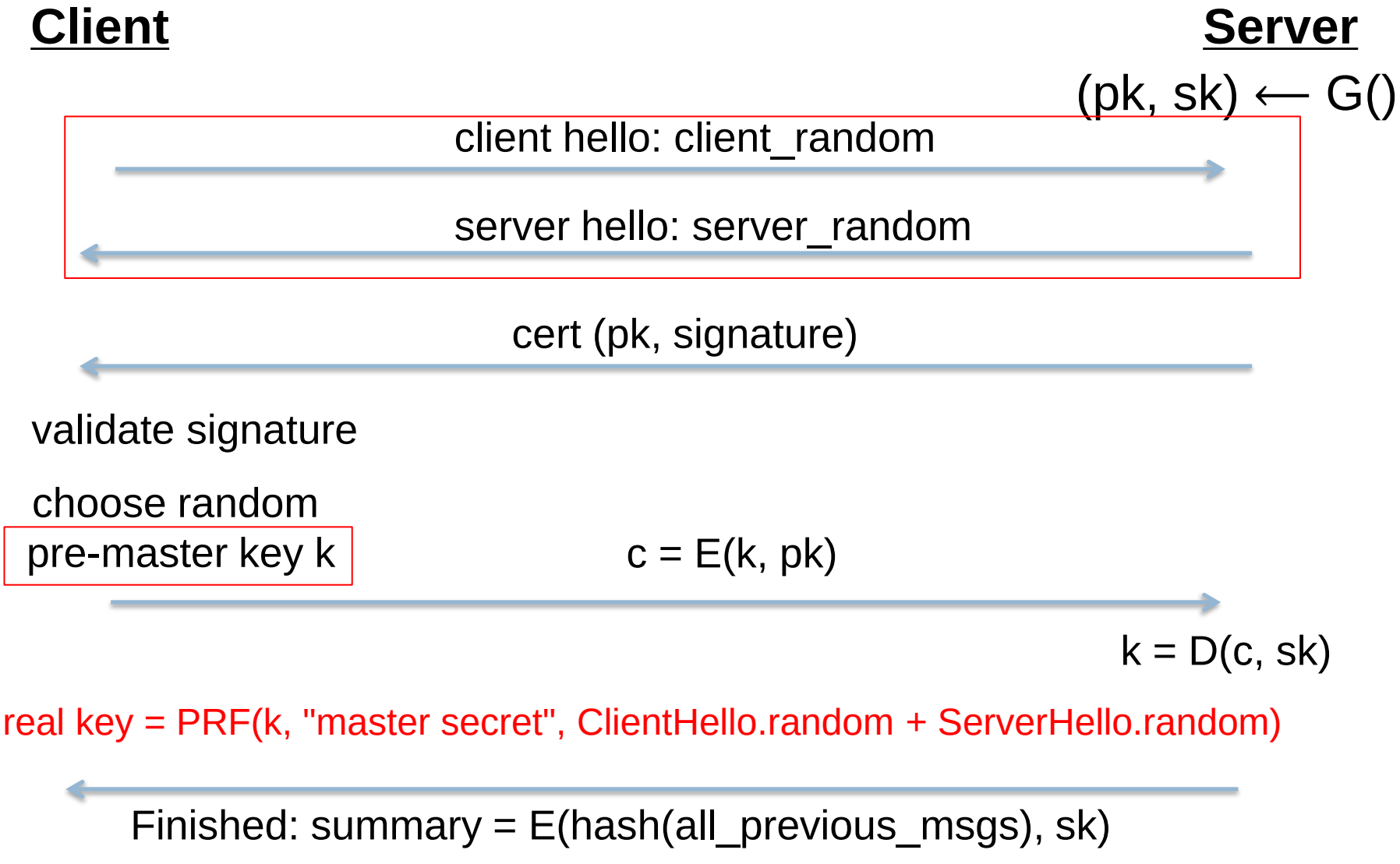
Client

Server

(pk, sk) ⟵ G()



Public Key Cryptography -- putting everything together – SSL/TLS



Relationship between HTTPS and SSL/TLS

- SSL/TLS is a generic cryptographic protocol. SSL initially, later TLS (successor).

- HTTPS uses SSL/TLS
- SSH uses SSL/TLS as well

Some Practical Advice

- **HMAC:** *HMAC-SHA256*
- **Block Cipher:** *AES-256*
- **Randomness:** OS Cryptographic Pseudo Random Number Generator (CPRNG)
- **Public Key Encryption:** *RSA* or *ECDSA*
- **Implementation:** *OpenSSL*

Don’t Roll Your Own!!